Projektni zadatak 2
VVS

Tim:	Brojevi indeksa:
Ismail Džafić	19588
Tarik Lolić	19513
Eldar Hadžiselimović	19434
Faruk Avdagić	19350

Contents

1 McCabe metrika, kontrolni graf **6**

1.1 AnalizirajObavjestenja . . . 6

1.2 AnalizirajPerformanseNarudzbi . . . 8

1.3 AnalizirajStanjeInventara . . . 10

1.4 AnalizirajDobavljaca . . . 12

2 White Box Testiranje **14**

2.1 AnalizirajObavjestenja . . . 14

2.1.1 Test 1: Prazan Skup (Branch Coverage) . . . 14

2.1.2 Test 2: Nema na Stanju (Condition Coverage) . . . 14

2.1.3 Test 3: Niske Zalihe (Condition/Boundary Coverage) . . . 15

2.1.4 Test 4: Potrebna Narudžba (Condition Coverage) . . . 15

2.1.5 Test 5: Sumarni Izvještaj - Kritično (Branch Coverage) . . . 15

2.1.6 Test 6: Sumarni Izvještaj - Srednji Prioritet (Branch Coverage) . . . 15

2.1.7 Test 7: Vremenska Analiza (Switching Logic) . . . 16

2.1.8 Test 8: Statistička Analiza (Switching Logic) . . . 16

2.1.9 Test 9: Akcioni Plan (Switching Logic) . . . 16

2.1.10 Test 10: Upozorenje o Hitnosti (Boundary Ratio Analysis) . . . 16

2.1.11 Test 11: Osnovna Putanja (Path Coverage) . . . 17

2.1.12 Test 12: Zastarjela Obavještenja (Deep Condition Logic) . . . 17

2.2 AnalizirajPerformanseNarudzbi . . . 17

2.2.1 Test 1: Nema Narudžbi (Branch Coverage) . . . 17

2.2.2 Test 2: Status Na Čekanju (Loop/Condition Coverage) . . . 17

2.2.3 Test 3: Status Isporučeno (Loop/Condition Coverage) . . . 18

2.2.4 Test 4: Status Otkazano (Loop/Condition Coverage) . . . 18

2.2.5 Test 5: Kritično Čekanje (Boundary Analysis) . . . 18

2.2.6 Test 6: Visok Postotak Otkazivanja (Boundary Analysis) . . . 18

2.2.7 Test 7: Vremenska Analiza (Toggle Coverage) . . . 19

2.2.8 Test 8: Finansijska Analiza (Toggle Coverage) . . . 19

2.2.9 Test 9: Detaljni Izvještaj (Toggle Coverage) . . . 19

2.2.10 Test 10: Nizak Stepen Isporuke (Boundary Condition) . . . 19

2.2.11 Test 11: Uspješna Realizacija (Path Coverage) . . . 20

2.2.12 Test 12: Zamrznuti Kapital (Financial Logic) . . . 20

2.3 AnalizirajStanjeInventara . . . 20

2.3.1 Test 1: Prazan Inventar (Branch Coverage) . . . 20

2.3.2 Test 2: Kritična Zaliha (Condition Coverage) . . . 20

2.3.3 Test 3: Niska Zaliha (Condition/Boundary) . . . 21

2.3.4 Test 4: Zdrava Zaliha (Condition Coverage) . . . 21

2.3.5 Test 5: Sumarni Izvještaj - Kritično (Branch Coverage) . . . 21

2.3.6 Test 6: Sumarni Izvještaj - Nisko (Branch Coverage) . . . 21

2.3.7 Test 7: Kategorije (Toggle Coverage) . . . 22

2.3.8 Test 8: Dobavljači (Toggle Coverage) . . . 22

2.3.9 Test 9: Detaljna Analiza (Toggle Coverage) . . . 22

2.3.10 Test 10: Upozorenje o Zdravlju Inventara (Boundary Ratio) . . . 22

2.3.11 Test 11: Osnovna Putanja (Path Coverage) . . . 22
2.4 AnalizirajDobavljaca . . . 23
2.4.1 Test 1: Dobavljač ne postoji (Branch Coverage) . . . 23
2.4.2 Test 2: Naziv Prekratak (Boundary Analysis) . . . 23
2.4.3 Test 3: Naziv Predugačak (Boundary Analysis) . . . 23
2.4.4 Test 4: Kontakt Nedostaje (Branch Coverage) . . . 24
2.4.5 Test 5: Email Format (Path Coverage) . . . 24
2.4.6 Test 6: Telefon Format (Path Coverage) . . . 24
2.4.7 Test 7: Nepoznat Format (Path Coverage) . . . 24
2.4.8 Test 8: Nema Duplikata (Loop Testing) . . . 25
2.4.9 Test 9: Pronađen Duplikat (Loop Testing) . . . 25
2.4.10 Test 10: Visok Kvalitet (Condition Coverage) . . . 25
2.4.11 Test 11: Srednji Kvalitet (Condition Coverage) . . . 25
2.4.12 Test 12: Robusnost (Exception Handling) . . . 26

3 Code Tuning **27**
3.1 AnalizirajObavjestenja . . . 29
3.1.1 StringBuilder . . . 29
3.1.2 Keširanje Count . . . 30
3.1.3 List Capacity . . . 31
3.2 AnalizirajPerformanseNarudzbi . . . 31
3.2.1 Keširanje DateTime.Now . . . 31
3.2.2 Early exit . . . 32
3.2.3 Minimiziranje rada u petlji . . . 33
3.3 AnalizirajStanjeInventara . . . 34
3.3.1 Zamjena LINQ . . . 34
3.3.2 Eliminacija duplikata . . . 34
3.3.3 string.Join() . . . 35
3.4 AnalizirajDobavljaca . . . 36
3.4.1 Uklanjanje ToString() . . . 36
3.4.2 Kombinovanje uslova . . . 36
3.4.3 Lokalne varijable . . . 37
3.4.4 Krajnji izgled svih testova performansi: . . . 38

4 Refaktoring **38**
4.1 AnalizirajObavjestenja . . . 39
4.1.1 Kod prije refaktoringa . . . 39
4.1.2 Analiza primijenjenih checklista . . . 42
4.1.3 Kod poslije refaktoringa . . . 42
4.2 AnalizirajPerformanseNarudzbi . . . 45
4.2.1 Kod prije refaktoringa . . . 45
4.2.2 Analiza primijenjenih checklista . . . 47
4.2.3 Kod poslije refaktoringa . . . 47
4.3 AnalizirajStanjeInventara . . . 49
4.3.1 Kod prije refaktoringa . . . 49
4.3.2 Analiza primijenjenih checklista . . . 51

4.3.3 Kod poslije refaktoringa . . . 52
4.4 AnalizirajDobavljaca . . . 54
4.4.1 Kod prije refaktoringa . . . 54
4.4.2 Analiza primijenjenih checklista . . . 56
4.4.3 Kod poslije refaktoringa . . . 57
4.5 Code Metrics rezultati prije refaktoringa . . . 58
4.6 Code Metrics rezultati poslije refaktoringa . . . 59
4.7 Uporedni pregled metrika . . . 59
4.8 Analiza pojedinačnih metoda . . . 59
4.8.1 `AnalizirajDobavljaca(...)` . . . 59
4.8.2 `AnalizirajStanjeInventara(...)` . . . 59
4.8.3 `AnalizirajPerformanseNarudzbi(...)` . . . 60
4.8.4 `AnalizirajObavjestenja(...)` . . . 60
4.9 Zaključak . . . 60

Uvod

Prilikom analize neophodno je bilo izvršiti nekoliko ispravki na projektu kako bismo mogli na odgovarajući način uraditi drugi dio projektnog zadatka. Odabrane metode za analizu, koje zadovoljavaju postavljene uslove zadatka su:

a) AnalizirajObavjestenja

b) AnalizirajPerformanseNarudzbi

c) AnalizirajStanjeInventara

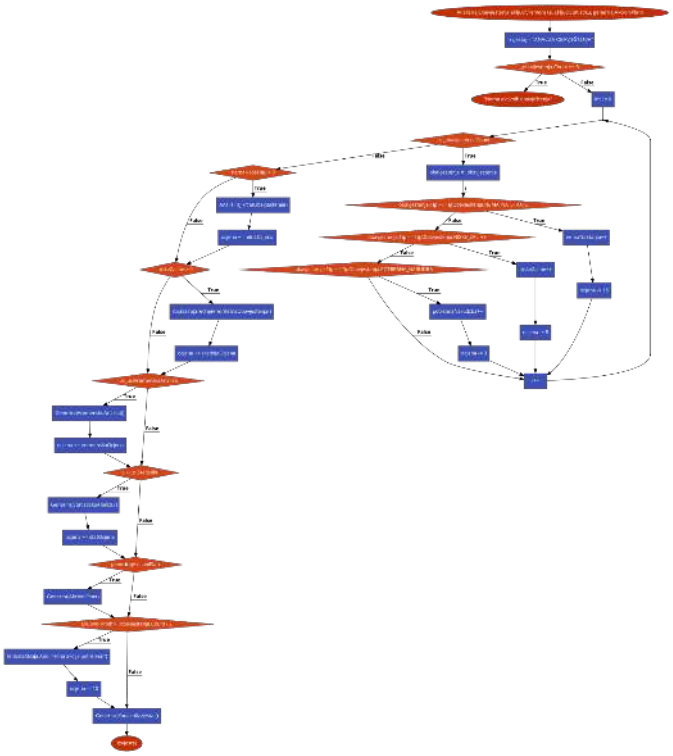
d) AnalizirajDobavljaca

Modifikovani kod se nalazi na grani *ProjektniKod*. Što se tiče dijela analize metoda, raspodjela je:

- AnalizirajObavjestenja - Eldar Hadžiselimović
- AnalizirajPerformanseNarudzbi - Ismail Džafić
- AnalizirajStanjeInventara - Faruk Avdagić
- AnalizirajDobavljaca - Tarik Lolić

1 McCabe metrika, kontrolni graf

1.1 AnalizirajObavjestenja



Kontrolni graf metode AnalizirajObavjestenja

Tačke odlučivanja u metodi AnalizirajObavjestenja se nalaze u sljedećim linijama koda:

Linija 7: `if (_obavjestenja.Count == 0)`

Linija 19: `for (int i = 0; i < _obavjestenja.Count; i++)`

Linija 22: `if (obavjestenje.Tip == TipObavjestenja.NEMA_NA_STANJU)`

Linija 27: `else if (obavjestenje.Tip == TipObavjestenja.NISKE_ZALIHE)`

Linija 32: `else if (obavjestenje.Tip == TipObavjestenja.POTREBNA_NARUDBA)`

Linija 45: `if (nemaNaStanju > 0)`

Linija 54: `if (niskeZalihe > 0)`

Linija 63: `if (ukljuciVremenskaAnaliza)`

Linija 73: `if (ukljuciStatistiku)`

Linija 83: `if (generirajAkcioniPlan)`

Linija 90: `if (ukupnoHitnih > _obavjestenja.Count / 2)`

što znači da će McCabe metrika biti $11 + 1 = 12$ kao što smo dobili i korištenjem Code Metrics alata.

Vrijednost 12 spada u kategoriju umjerene kompleksnosti, što ukazuje na povećan rizik i potrebu za pažljivijem testiranju. Metoda je prešla granicu niske kompleksnosti (10) i sada zahtijeva dodatnu pažnju pri održavanju.

Indeks održavanja koji se računa na osnovu Halstead metrike, ciklomatske kompleksnosti i broja linija koda iznosi 37 što spada u zelenu zonu, što znači da je kod relativno lako održiv. Međutim, postoji prostor za poboljšanje do optimalne vrijednosti (70+).

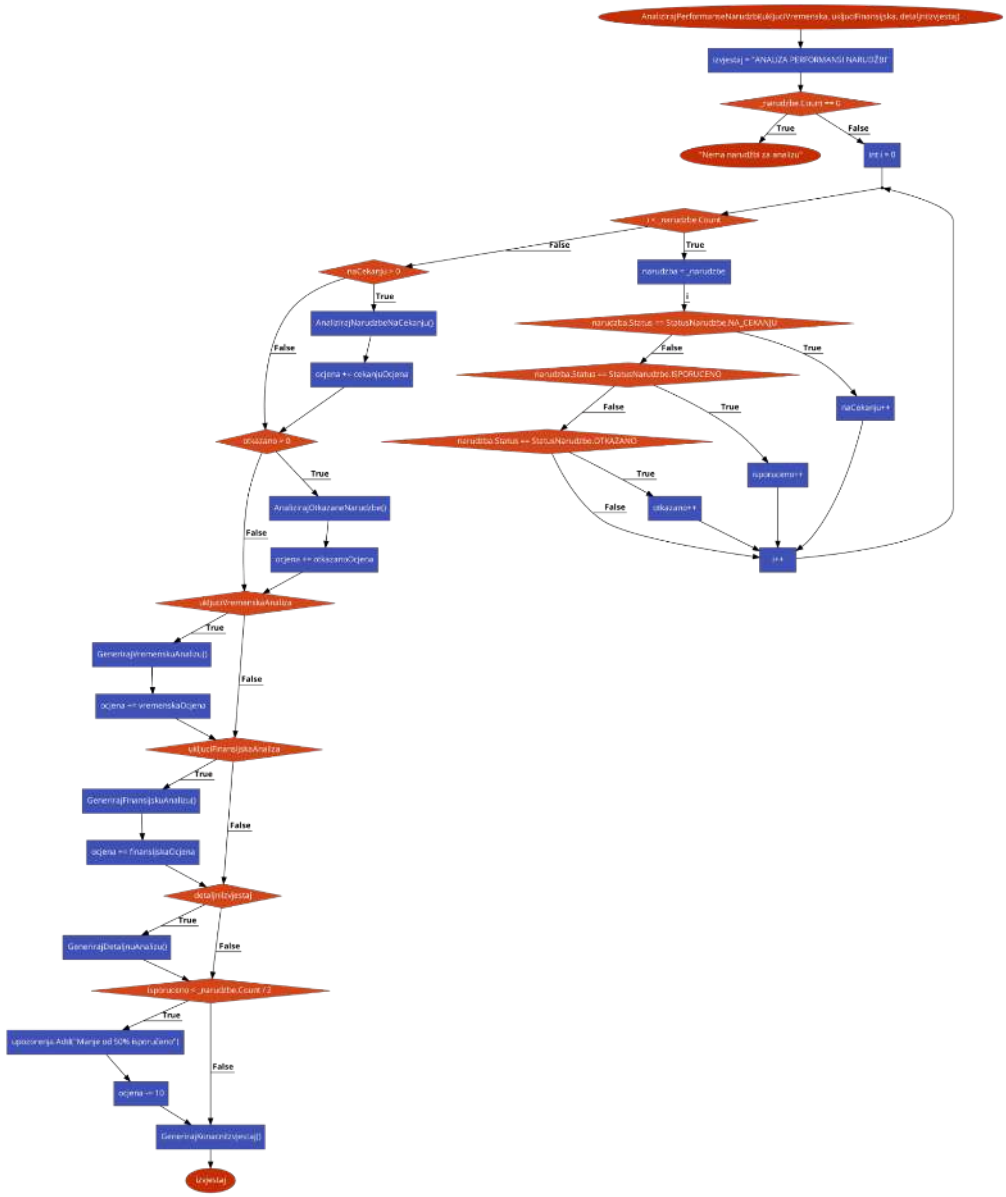
Tri bool(ukljuciVremenskaAnaliza, ukljuciStatistiku, generirajAkcioniPlan) direktno doprinose kompleksnosti sa 3 decision points ($2^3 = 8$). Ovo omogućava veliku fleksibilnost poziva metode, ali istovremeno povećava prostor testiranja i otežava predviđanje ponašanja metode.
Pozitivni aspekti:

- Koristi se early return pattern (rani izlaz iz metode kada nema obavještenja)
- Jasna sekvencijalna struktura sa logičkim grupama funkcionalnosti
- Korištenje tuple dekonstrukcije za povratne vrijednosti podfunkcija

Oblasti za poboljšanje:

- Dužina metode (preko 95 linija) otežava razumijevanje
- Mogućnost primjene Strategy pattern-a za različite vrste analiza
- For petlja sa ugniježđenim if-else strukturom povećava kompleksnost
- Inline logika za analizu po tipu mogla bi biti izdvojena u zasebnu metodu

1.2 AnalizirajPerformanseNarudzbi



Kontrolni graf metode AnalizirajPerformanseNarudzbi

Tačke odlučivanja u metodi AnalizirajPerformanseNarudzbi se nalaze u sljedećim linijama koda:

Linija 7: `if (_narudzbe.Count == 0)`

Linija 20: `for (int i = 0; i < _narudzbe.Count; i++)`

Linija 23: `if (narudzba.Status == StatusNarudzbe.NA_CEKANJU)`

Linija 27: `else if (narudzba.Status == StatusNarudzbe.ISPORUCENO)`

Linija 31: `else if (narudzba.Status == StatusNarudzbe.OTKAZANO)`

Linija 42: `if (naCekanju > 0)`

Linija 51: `if (otkazano > 0)`

Linija 60: `if (ukljuciVremenskaAnaliza)`

Linija 70: `if (ukljuciFinansijskaAnaliza)`

Linija 80: `if (detaljniIzvjestaj)`

Linija 86: `if (isporuceno < _narudzbe.Count / 2)`

što znači da će McCabe metrika biti 11+1=12 kao što smo dobili i korištenjem Code Metrics alata.

Vrijednost 12 spada u kategoriju umjerene kompleksnosti, što ukazuje na povećan rizik i potrebu za pažljivijem testiranju. Metoda je prešla granicu niske kompleksnosti (10) i sada zahtijeva dodatnu pažnju pri održavanju.

Indeks održavanja koji se računa na osnovu Halstead metrike, ciklomatske kompleksnosti i broja linija koda iznosi 38 što spada u zelenu zonu, što znači da je kod relativno lako održiv. Međutim, postoji prostor za poboljšanje do optimalne vrijednosti (70+).

Tri bool parametra (ukljuciVremenskaAnaliza, ukljuciFinansijskaAnaliza, detaljniIzvjestaj) direktno doprinose kompleksnosti sa 3 decision points ($2^3 = 8$). Ovo omogućava veliku fleksibilnost poziva metode, ali istovremeno povećava prostor testiranja i otežava predviđanje ponašanja metode.

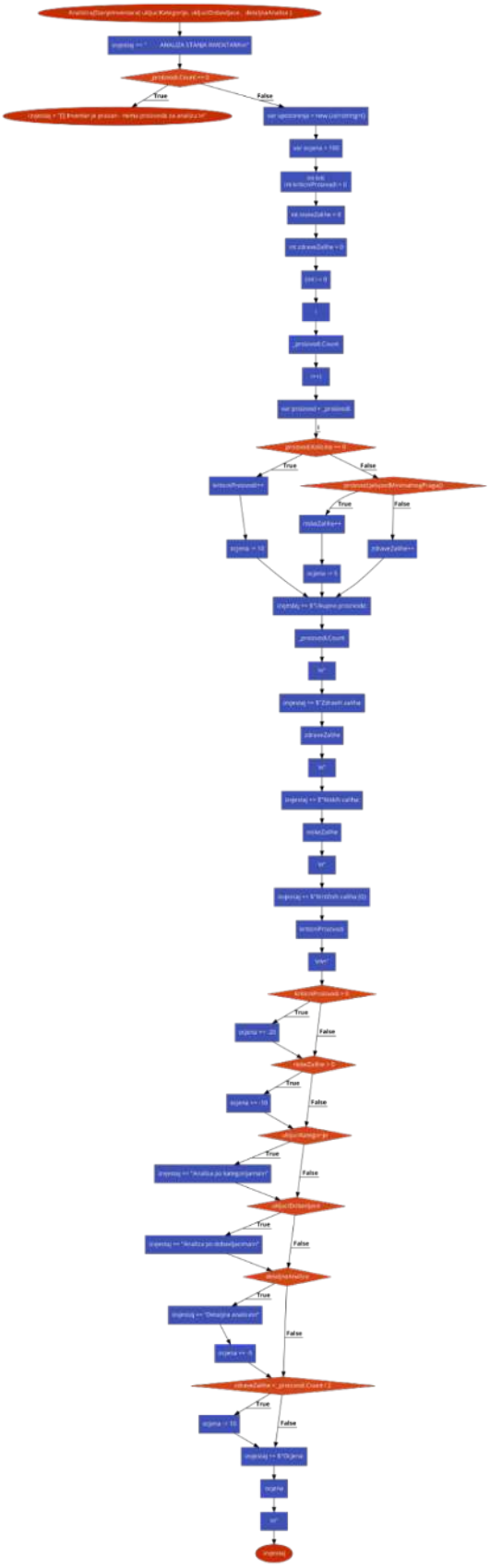
Pozitivni aspekti:

- Koristi se early return pattern (rani izlaz iz metode kada nema narudžbi)
- Jasna sekvencijalna struktura sa logičkim grupama funkcionalnosti
- Korištenje tuple dekonstrukcije za povratne vrijednosti podfunkcija

Oblasti za poboljšanje:

- Dužina metode (preko 90 linija) otežava razumijevanje
- Mogućnost primjene Strategy pattern-a za različite vrste analiza
- For petlja sa ugniježđenim if-else strukturom povećava kompleksnost
- Inline logika za analizu po statusu mogla bi biti izdvojena u zasebnu metodu

1.3 AnalizirajStanjeInventara



Kontrolni graf metode AnalizirajStanjeInventara

Tačke odlučivanja u metodi AnalizirajStanjeInventara se nalaze u sljedećim linijama koda:

Linija 7: `if (_proizvodi.Count == 0)`

Linija 20: `for (int i = 0; i < _proizvodi.Count; i++)`

Linija 23: `if (proizvod.Kolicina == 0)`

Linija 28: `else if (proizvod.JeIspodMinimalnogPraga())`

Linija 33: `else`

Linija 44: `if (kriticniProizvodi > 0)`

Linija 53: `if (niskeZalihe > 0)`

Linija 62: `if (ukljuciKategorije)`

Linija 68: `if (ukljuciDobavljace)`

Linija 74: `if (detaljnaAnaliza)`

Linija 84: `if (zdraveZalihe < _proizvodi.Count / 2)`

što znači da će McCabe metrika biti 11+1=12 kao što smo dobili i korištenjem Code Metrics alata.

Vrijednost 12 spada u kategoriju umjerene kompleksnosti, što ukazuje na povećan rizik i potrebu za pažljivijem testiranju. Metoda je prešla granicu niske kompleksnosti (10) i sada zahtijeva dodatnu pažnju pri održavanju.

Indeks održavanja koji se računa na osnovu Halstead metrike, ciklomatske kompleksnosti i broja linija koda iznosi 39 što spada u zelenu zonu, što znači da je kod relativno lako održiv. Međutim, postoji prostor za poboljšanje do optimalne vrijednosti (70+).

Tri bool parametra (ukljuciKategorije, ukljuciDobavljace, detaljnaAnaliza) direktno doprinose kompleksnosti sa 3 decision points ($2^3 = 8$). Ovo omogućava veliku fleksibilnost poziva metode, ali istovremeno povećava prostor testiranja i otežava predviđanje ponašanja metode.

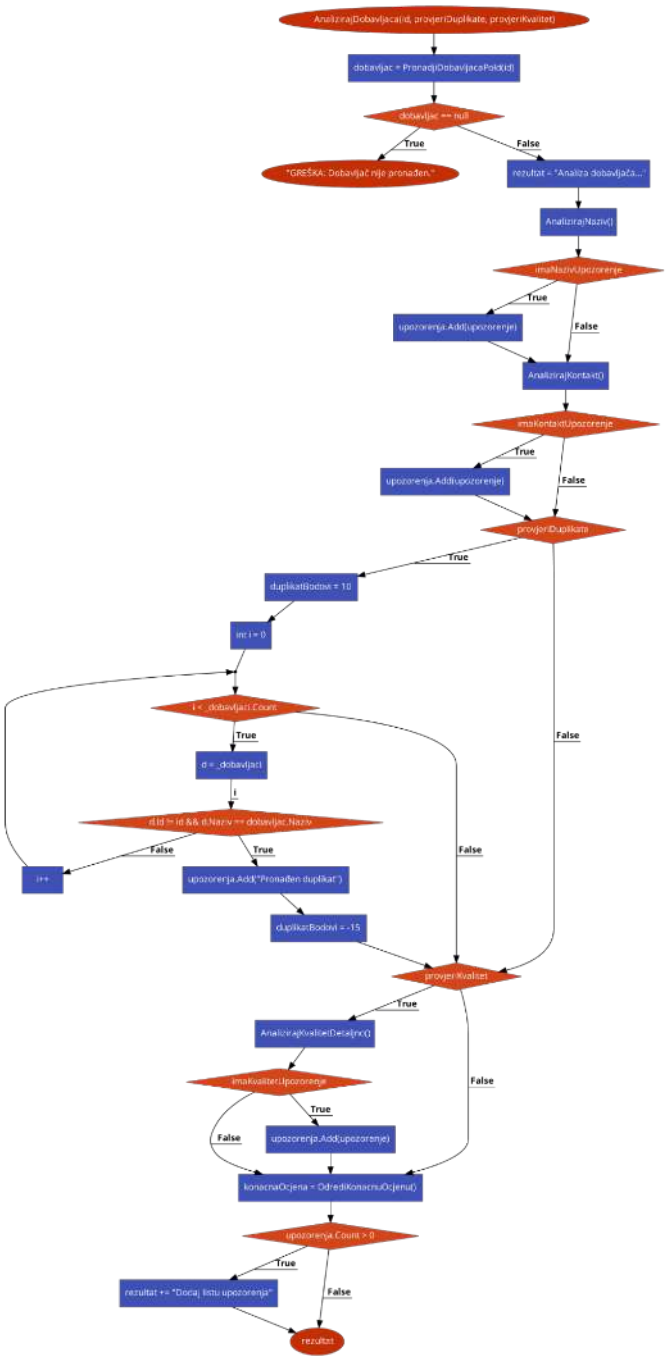
Pozitivni aspekti:

- Koristi se early return pattern (rani izlaz iz metode kada je inventar prazan)
- Jasna sekvencijalna struktura sa logičkim grupama funkcionalnosti
- Korištenje tuple dekonstrukcije za povratne vrijednosti podfunkcija

Oblasti za poboljšanje:

- Dužina metode (preko 85 linija) otežava razumijevanje
- Mogućnost primjene Strategy pattern-a za različite vrste analiza
- For petlja sa ugniježđenim if-else-else strukturom povećava kompleksnost
- Inline logika za analizu proizvoda mogla bi biti izdvojena u zasebnu metodu

1.4 AnalizirajDobavljaca



Kontrolni graf metode AnalizirajDobavljaca

Tačke odlučivanja u metodi AnalizirajDobavljaca se nalaze u sljedećim linijama koda:

Linija 3: `if (dobavljac == null)`

Linija 13: `if (!string.IsNullOrEmpty(nazivUpozorenje))`

Linija 20: `if (!string.IsNullOrEmpty(kontaktUpozorenje))`

Linija 26: `if (provjeriDuplikate)`

Linija 30: `for (int i = 0; i < _dobavljaci.Count; i++)`

Linija 33: `if (d.Id != id && d.Naziv.Equals(dobavljac.Naziv, StringComparison.OrdinalIgnoreCase) ↪ )`

Linija 42: `if (provjeriKvalitet)`

Linija 53: `if (upozorenja.Count > 0)`

što znači da će McCabe metrika biti 8+1=9 kao što smo dobili i korištenjem Code Metrics alata.

Vrijednost 9 spada u kategoriju niske kompleksnosti, što je pozitivan pokazatelj. Međutim, metoda se nalazi na gornjoj granici ovog ranga, što ukazuje da bi daljnje dodavanje uslova moglo dovesti do prelaska u kategoriju umjerene kompleksnosti.

Indeks održavanja koji se računa na osnovu Halstead metrike, ciklomatske kompleksnosti i broja linija koda iznosi 43 što spada u zelenu zonu, što znači da je kod relativno lako održiv. Međutim, postoji prostor za poboljšanje do optimalne vrijednosti (70+).

Dva bool parametra (provjeriDuplikate, provjeriKvalitet) direktno doprinose kompleksnosti sa 2 decision points ($2^3 = 4$). Ovo omogućava veliku fleksibilnost poziva metode, ali istovremeno povećava prostor testiranja i otežava predviđanje ponašanja metode.

Pozitivni aspekti:

- Koristi se early return pattern (rani izlaz iz metode kada dobavljač nije pronađen)
- Jasna sekvencijalna struktura sa logičkim grupama funkcionalnosti
- Korištenje tuple dekonstrukcije za povratne vrijednosti podfunkcija

Oblasti za poboljšanje:

- Dužina metode (preko 55 linija) otežava razumijevanje
- For petlja sa ugniježđenim if uslovom povećava kompleksnost
- Inline logika za provjeru duplikata mogla bi biti izdvojena u zasebnu metodu
- Mogućnost primjene Strategy pattern-a za različite vrste provjera

2 White Box Testiranje

Proces testiranja "bijele kutije" proveden je uz pomoć alata **Fine Code Coverage** za analizu pokrivenosti koda. Prikaz početnog stanja pokrivenosti, prije implementacije dodatnih testova.

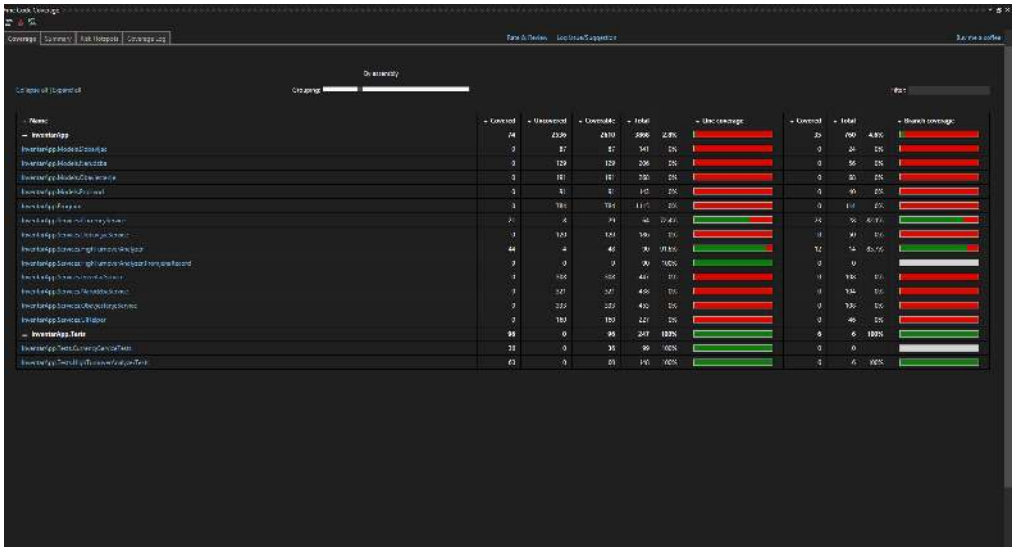


Figure 1: Code Coverage Prije Pisanja Testova

McCabe metrika (ciklomatska kompleksnost) mjeri broj linearno nezavisnih puteva kroz kod, što direktno indicira minimalan broj testova potrebnih za potpunu pokrivenost grana

Happy path (sretna putanja) predstavlja scenario u kojem se ulazni podaci biraju tako da program izvrši svoju primarnu funkciju bez grešaka, izuzetaka ili aktivacije grana za rukovanje nevalidnim podacima. Fokus je na potvrdi da sistem ispravno obrađuje standardne, očekivane slučajeve koji predstavljaju srž poslovne logike. Ovaj pojam je ovdje samo objašnjen iz razloga što se spominje u dijelovima koji slijede.

2.1 AnalizirajObavjestenja

Primijenjene tehnike: Branch Coverage, Condition Coverage, Loop Testing, Boundary Value Analysis.

2.1.1 Test 1: Prazan Skup (Branch Coverage)

```
var result = _service.AnalizirajObavjestenja();
result.Should().Contain("Nema aktivnih obavještenja");
```

Inicijalno stanje liste obavještenja je prazno. Ovim se direktno zadovoljava "Guard Clause" uslov `if (_obavjestenja.Count == 0)`, verifikujući da metoda odmah prekida izvršavanje i vraća odgovarajuću poruku bez nepotrebnog ulaska u petlje.

2.1.2 Test 2: Nema na Stanju (Condition Coverage)

```
var proizvod = new Proizvod("P1", 0, 5, "D1", KategorijaProizvoda.ALATI);
_service.KreirajObavjestenje(proizvod);
```

```
var result = _service.AnalizirajObavjestenja(false, false, false);
result.Should().Contain("Nema na stanju: 1");
```

Varijabla `kolicina` je postavljena na `0` (nula). Ova vrijednost je odabrana kako bi zadovoljila uslov `if (obavjestenje.Tip == TipObavjestenja.NEMA_NA_STANJU)`, koji se aktivira kada je količina manja od 1. Test potvrđuje ispravnu detekciju kritičnog nedostatka zaliha.

2.1.3 Test 3: Niske Zalihe (Condition/Boundary Coverage)

```
var proizvod = new Proizvod("P1", 2, 5, "D1", KategorijaProizvoda.ALATI);
_service.KreirajObavjestenje(proizvod);
var result = _service.AnalizirajObavjestenja(false, false, false);
result.Should().Contain("Niske zalihe: 1");
```

Količina je `2`, a minimalni prag `5`. Vrijednost 2 je odabrana jer je veća od 0 (nije prazno) ali manja od 5, čime se cilja `else if` grana za `TipObavjestenja.NISKE_ZALIHE`. Ovo verifikuje logiku upozorenja za dopunu zaliha.

2.1.4 Test 4: Potrebna Narudžba (Condition Coverage)

```
_service.KreirajObavjestenjeZaNarudzbu("P1", "Order needed");
var result = _service.AnalizirajObavjestenja(false, false, false);
result.Should().Contain("Potrebna narudžba: 1");
```

Kreirano je obavještenje eksplicitno vegazno za narudžbu. Ovim se testira treća grana unutar petlje (`TipObavjestenja.POTREB` osiguravajući da se i administrativna obavještenja ispravno broje.

2.1.5 Test 5: Sumarni Izvještaj - Kritično (Branch Coverage)

```
var proizvod = new Proizvod("P1", 0, 5, "D1", KategorijaProizvoda.ALATI);
_service.KreirajObavjestenje(proizvod);
var result = _service.AnalizirajObavjestenja(false, false, false);
result.Should().Contain("obavještenja je za proizvode bez zaliha");
```

Sa jednim proizvodom količine 0, brojač `nemaNaStanju` postaje 1. Ovo zadovoljava uslov `if (nemaNaStanju > 0)` izvan petlje, aktivirajući generisanje sekcije za hitna obavještenja.

2.1.6 Test 6: Sumarni Izvještaj - Srednji Prioritet (Branch Coverage)

```
var proizvod = new Proizvod("P1", 2, 5, "D1", KategorijaProizvoda.ALATI);
_service.KreirajObavjestenje(proizvod);
var result = _service.AnalizirajObavjestenja(false, false, false);
result.Should().Contain("proizvoda sa niskim zalihama");
```

Sa proizvodom količine 2, brojač `niskeZalihe` postaje 1, čime se aktivira nezavisni `if (niskeZalihe > 0)` blok za generisanje upozorenja srednjeg prioriteta.

2.1.7 Test 7: Vremenska Analiza (Switching Logic)

```
_service.KreirajObavjestenjeZaNarudzbu("P1", "Test");
var result = _service.AnalizirajObavjestenja(ukljuciVremenskaAnaliza: true, false, false)
    ↪ ;
result.Should().Contain("Vremenska analiza");
```

Booleov parametar `ukljuciVremenskaAnaliza` je postavljen na `true`. Ovo je jedini način da se uđe u `GenerirajVremensku` metodu, čime se testira modularnost izvještaja.

2.1.8 Test 8: Statistička Analiza (Switching Logic)

```
_service.KreirajObavjestenjeZaNarudzbu("P1", "Test");
var result = _service.AnalizirajObavjestenja(false, ukljuciStatistiku: true, false);
result.Should().Contain("Statistička analiza");
```

Parametar `ukljuciStatistiku` je aktivan. Ovo verifikuje granu koja poziva `GenerirajStatistiku`, osiguravajući da se statistički podaci prikazuju samo na zahtjev.

2.1.9 Test 9: Akcioni Plan (Switching Logic)

```
_service.KreirajObavjestenjeZaNarudzbu("P1", "Test");
var result = _service.AnalizirajObavjestenja(false, false, generirajAkcioniPlan: true);
result.Should().Contain("Akcioni plan");
```

Aktiviranjem `generirajAkcioniPlan`, testira se poziv metode `GenerirajAkcioniPlan`, potvrđujući da korisnik može dobiti preporuke za akciju.

2.1.10 Test 10: Upozorenje o Hitnosti (Boundary Ratio Analysis)

```
_service.KreirajObavjestenje(new Proizvod("P1", 0, 5, "D1", KategorijaProizvoda.ALATI));
_service.KreirajObavjestenje(new Proizvod("P2", 2, 5, "D1", KategorijaProizvoda.ALATI));
var result = _service.AnalizirajObavjestenja(false, false, false);
result.Should().Contain("Više od 50% obavještenja zahtijeva hitnu akciju");
```

Unesena su dva proizvoda koja oba spadaju u hitnu kategoriju (jedan "Nema na stanju", jedan "Niske zalihe"). Ukupno 2, hitnih 2. Uslov `ukupnoHitnih > Count / 2` ($2 > 1$) je zadovoljen. Ovo testira logiku koja alarmira korisnika kada je većina inventara u problemu.

2.1.11 Test 11: Osnovna Putanja (Path Coverage)

```
_service.KreirajObavjestenjeZaNarudzbu("P1", "Normal");
_service.KreirajObavjestenjeZaNarudzbu("P2", "Normal");
var result = _service.AnalizirajObavjestenja(false, false, false);
result.Should().NotContain("hitnu akciju");
```

Happy path scenario: postoje obavještenja, ali nisu hitna, i sve dodatne opcije su isključene. Ovo potvrđuje da se upozorenja NE generišu kada sistem radi normalno.

2.1.12 Test 12: Zastarjela Obavještenja (Deep Condition Logic)

```
_service.KreirajObavjestenjeZaNarudzbu("Old", "Outdated");
var all = _service.PrikaziSvaObavjestenja();
all[0].VrijemeKreiranja = DateTime.Now.AddDays(-10);
var result = _service.AnalizirajObavjestenja(ukljuciVremenskaAnaliza: true, false, false)
    ↪ ;
result.Should().Contain("KRITIČNO: 1 obavještenja nije riješeno više od 7 dana");
```

Vrijeme kreiranja je ručno postavljeno na 10 dana u prošlost. Ovo cilja uslov `DateTime.Now - o.VrijemeKreiranja > 7 dana` unutar vremenske analize, verifikujući detekciju starih, neriješenih problema.

2.2 AnalizirajPerformanseNarudzbi

Primijenjene tehnike: Branch Coverage, Condition Coverage, State Transition Testing.

2.2.1 Test 1: Nema Narudžbi (Branch Coverage)

```
var service = CreateService();
var result = service.AnalizirajPerformanseNarudzbi();
result.Should().Contain("Nema narudžbi za analizu");
```

Prazna lista narudžbi aktivira guard clause `if (_narudzbe.Count == 0)`, sprječavajući dalje izvršavanje i potencijalne greške dijeljenja s nulom.

2.2.2 Test 2: Status Na Čekanju (Loop/Condition Coverage)

```
var service = CreateService();
service.KreirajNarudzbu("P1", "D1", 10);
var result = service.AnalizirajPerformanseNarudzbi(false, false, false);
result.Should().Contain("Na čekanju: 1");
```

Nova narudžba podrazumijevano ima status `NA_CEKANJU`. Ovim se testira prva grana `if (Status == StatusNarudzbe.NA_` unutar petlje za analizu.

2.2.3 Test 3: Status Isporučeno (Loop/Condition Coverage)

```
var service = CreateService();
service.KreirajNarudzbu("P1", "D1", 10);
var id = service.PrikaziSveNarudzbe()[0].NarudzbaId;
service.OznaciKaoIsporuceno(id);
var result = service.AnalizirajPerformanseNarudzbi(false, false, false);
result.Should().Contain("Isporučeno: 1");
```

Promjenom statusa u `ISPORUCENO`, ovim testom ciljamo `else if (Status == StatusNarudzbe.ISPORUCENO)` granu, verificirajući da se uspješne narudžbe ispravno broje.

2.2.4 Test 4: Status Otkazano (Loop/Condition Coverage)

```
var service = CreateService();
service.KreirajNarudzbu("P1", "D1", 10);
var id = service.PrikaziSveNarudzbe()[0].NarudzbaId;
service.OtkaziNarudzbu(id);
var result = service.AnalizirajPerformanseNarudzbi(false, false, false);
result.Should().Contain("Otkazano: 1");
```

Otkazivanje narudžbe aktivira treću granu `else if (Status == StatusNarudzbe.OTKAZANO)`, osiguravajući da se i neuspješne transakcije evidentiraju.

2.2.5 Test 5: Kritično Čekanje (Boundary Analysis)

```
var service = CreateService();
service.KreirajNarudzbu("P1", "D1", 10);
var result = service.AnalizirajPerformanseNarudzbi(false, false, false);
result.Should().Contain("KRITIČNO: Više od 60% narudžbi je na čekanju");
```

Sa 1 od 1 narudžbe na čekanju (100%), prelazimo prag od 60% definiran u uslovu `if (postotakNaCekanju > 60)`. Ovo potvrđuje da sistem alarmira na zagušenje.

2.2.6 Test 6: Visok Postotak Otkazivanja (Boundary Analysis)

```
var service = CreateService();
service.KreirajNarudzbu("P1", "D1", 10);
service.OtkaziNarudzbu(service.PrikaziSveNarudzbe()[0].NarudzbaId);
var result = service.AnalizirajPerformanseNarudzbi(false, false, false);
result.Should().Contain("KRITIČNO: Više od 30% narudžbi je otkazano");
```

100% otkazanih narudžbi (1/1) prelazi prag od 30% (`if (postotakOtkazanih > 30)`), čime se verifikuje logika za detekciju problema sa realizacijom.

2.2.7 Test 7: Vremenska Analiza (Toggle Coverage)

```
var service = CreateService();
service.KreirajNarudzbu("P1", "D1", 10);
var result = service.AnalizirajPerformanseNarudzbi(ukljuciVremenskaAnaliza: true, false,
    ↪ false);
result.Should().Contain("Vremenska analiza");
```

Postavljanje `ukljuciVremenskaAnaliza = true` prisiljava izvršavanje `GenerirajVremenskuAnalizu` metode.

2.2.8 Test 8: Finansijska Analiza (Toggle Coverage)

```
var service = CreateService();
service.KreirajNarudzbu("P1", "D1", 10);
var result = service.AnalizirajPerformanseNarudzbi(false, ukljuciFinansijskaAnaliza: true
    ↪ , false);
result.Should().Contain("Finansijska analiza");
```

Postavljanje `ukljuciFinansijskaAnaliza = true` testira integraciju sa metodom za proračun vrijednosti narudžbi.

2.2.9 Test 9: Detaljni Izvještaj (Toggle Coverage)

```
var service = CreateService();
service.KreirajNarudzbu("P1", "D1", 10);
var result = service.AnalizirajPerformanseNarudzbi(false, false, detaljniIzvjestaj: true)
    ↪ ;
result.Should().Contain("Analiza po dobavljačima");
```

Aktivacija `detaljniIzvjestaj` osigurava da se generiše dodatna sekcija specifična za dobavljače.

2.2.10 Test 10: Nizak Stepen Isporuke (Boundary Condition)

```
var service = CreateService();
service.KreirajNarudzbu("Delivered1", "D1", 10);
service.OznaciKaoIsporuceno(service.PrikaziSveNarudzbe()[0].NarudzbaId);
service.KreirajNarudzbu("Pending1", "D1", 10);
service.KreirajNarudzbu("Pending2", "D1", 10);
service.KreirajNarudzbu("Pending3", "D1", 10);
var result = service.AnalizirajPerformanseNarudzbi(false, false, false);
result.Should().Contain("Manje od 50% narudžbi je isporučeno");
```

Sa 1 isporučenom narudžbom od ukupno 4 (25%), zadovoljava se uslov `if (isporuceno < Count / 2)` ($1 < 2$). Ovo je ključan test za detekciju neefikasnosti procesa isporuke.

2.2.11 Test 11: Uspješna Realizacija (Path Coverage)

```
var service = CreateService();
service.KreirajNarudzbu("Del1", "D1", 10);
service.KreirajNarudzbu("Del2", "D1", 10);
var all = service.PrikaziSveNarudzbe();
service.OznaciKaoIsporuceno(all[0].NarudzbaId);
service.OznaciKaoIsporuceno(all[1].NarudzbaId);
var result = service.AnalizirajPerformanseNarudzbi(false, false, false);
result.Should().Contain("Isporučeno: 2");
result.Should().NotContain("UPOZORENJA");
```

Scenario gdje je sve isporučeno (2/2). Potvrđuje da sistem ne generiše lažna upozorenja kada su performanse optimalne.

2.2.12 Test 12: Zamrznuti Kapital (Financial Logic)

```
var service = CreateService();
service.KreirajNarudzbu("BigPending", "D1", 100);
service.KreirajNarudzbu("SmallDelivered", "D1", 10);
service.OznaciKaoIsporuceno(service.PrikaziSveNarudzbe()[1].NarudzbaId);
var result = service.AnalizirajPerformanseNarudzbi(false, ukljuciFinansijskaAnaliza: true
    ↪ , false);
result.Should().Contain("kapitala je zamrznuto");
```

Kreiran je disbalans vrijednosti: velika narudžba (100) čeka, mala (10) je isporučena. Iako je 50% narudžbi isporučeno brojčano, vrijednosno je većina zamrznuta. Ovo testira `if (postotakZamrznutog > 50)` unutar finansijske analize.

2.3 AnalizirajStanjeInventara

Primijenjene tehnike: Branch Coverage, Condition Coverage, Loop Testing, Boundary Value Analysis.

2.3.1 Test 1: Prazan Inventar (Branch Coverage)

```
var service = CreateService();
var result = service.AnalizirajStanjeInventara();
result.Should().Contain("Inventar je prazan");
```

Prazna lista proizvoda aktivira uslov `if (_proizvodi.Count == 0)`, verifikujući da se analiza ne pokreće bespotrebno.

2.3.2 Test 2: Kritična Zaliha (Condition Coverage)

```
var service = CreateService();
service.DodajProizvod("Crit", 0, 5, "D1", KategorijaProizvoda.ALATI);
```

```
var result = service.AnalizirajStanjeInventara(false, false, false);
result.Should().Contain("Kritičnih zaliha (0): 1");
```

Proizvod sa količinom 0 zadovoljava uslov `if (proizvod.Kolicina == 0)` unutar petlje, što je prvi kriterij za klasifikaciju zdravlja zaliha.

2.3.3 Test 3: Niska Zaliha (Condition/Boundary)

```
var service = CreateService();
service.DodajProizvod("Low", 2, 5, "D1", KategorijaProizvoda.ALATI);
var result = service.AnalizirajStanjeInventara(false, false, false);
result.Should().Contain("Niskih zaliha: 1");
```

Količina 2 je manja od minimalnog praga 5, ali veća od 0. Ovim se cilja `else if (proizvod.JeIspodMinimalnogPraga())` grana, testirajući upozorenja o potrebi dopune.

2.3.4 Test 4: Zdrava Zaliha (Condition Coverage)

```
var service = CreateService();
service.DodajProizvod("Good", 10, 5, "D1", KategorijaProizvoda.ALATI);
var result = service.AnalizirajStanjeInventara(false, false, false);
result.Should().Contain("Zdravih zaliha: 1");
```

Količina 10 je značajno iznad praga 5. Ovo verifikuje `else` granu ("Sretni put"), osiguravajući da se zdrav inventar ispravno klasificira.

2.3.5 Test 5: Sumarni Izvještaj - Kritično (Branch Coverage)

```
var service = CreateService();
service.DodajProizvod("Crit", 0, 5, "D1", KategorijaProizvoda.ALATI);
var result = service.AnalizirajStanjeInventara(false, false, false);
result.Should().Contain("bez zaliha!");
```

Prisustvo proizvoda sa 0 zaliha inkrementira `kriticniProizvodi` brojač, što kasnije aktivira `if (kriticniProizvodi > 0)` blok za generisanje alarma.

2.3.6 Test 6: Sumarni Izvještaj - Nisko (Branch Coverage)

```
var service = CreateService();
service.DodajProizvod("Low", 2, 5, "D1", KategorijaProizvoda.ALATI);
var result = service.AnalizirajStanjeInventara(false, false, false);
result.Should().Contain("ima niske zalihe");
```

Prisustvo proizvoda sa niskim zalihama (ali > 0) aktivira `if (niskeZalihe > 0)` blok.

2.3.7 Test 7: Kategorije (Toggle Coverage)

```
var service = CreateService();
service.DodajProizvod("P1", 10, 5, "D1", KategorijaProizvoda.IT_UREDJAJI);
var result = service.AnalizirajStanjeInventara(ukljuciKategorije: true, false, false);
result.Should().Contain("IT_UREDJAJI");
```

Aktivacija `ukljuciKategorije` verifikuje poziv metode za grupisanje po kategorijama.

2.3.8 Test 8: Dobavljači (Toggle Coverage)

```
var service = CreateService();
service.DodajProizvod("P1", 10, 5, "DobavljacXYZ", KategorijaProizvoda.ALATI);
var result = service.AnalizirajStanjeInventara(false, ukljuciDobavljace: true, false);
result.Should().Contain("Dobavljač DobavljacXYZ");
```

Testira integraciju analize po dobavljačima kroz odgovarajuću zastavicu.

2.3.9 Test 9: Detaljna Analiza (Toggle Coverage)

```
var service = CreateService();
service.DodajProizvod("P1", 10, 5, "D1", KategorijaProizvoda.ALATI);
var result = service.AnalizirajStanjeInventara(false, false, detaljnaAnaliza: true);
result.Should().Contain("Prosječna količina");
```

Verifikuje generisanje dodatnih statističkih metrika kada se zatraži detaljna analiza.

2.3.10 Test 10: Upozorenje o Zdravlju Inventara (Boundary Ratio)

```
var service = CreateService();
service.DodajProizvod("Bad1", 0, 5, "D1", KategorijaProizvoda.ALATI);
service.DodajProizvod("Bad2", 0, 5, "D1", KategorijaProizvoda.ALATI);
service.DodajProizvod("Bad3", 0, 5, "D1", KategorijaProizvoda.ALATI);
service.DodajProizvod("Good", 10, 5, "D1", KategorijaProizvoda.ALATI);
var result = service.AnalizirajStanjeInventara(false, false, false);
result.Should().Contain("Manje od 50% proizvoda ima zdrave zalihe");
```

Kreiran je scenario gdje je samo 1 od 4 proizvoda zdrav. Uslov upozorenja je `zdraveZalihe < Count / 2`. 1 je manje od 2 (4/2), čime se potvrđuje aktiviranje upozorenja.

2.3.11 Test 11: Osnovna Putanja (Path Coverage)

```
var service = CreateService();
service.DodajProizvod("Good1", 10, 5, "D1", KategorijaProizvoda.ALATI);
```

```
service.DodajProizvod("Good2", 10, 5, "D1", KategorijaProizvoda.ALATI);
var result = service.AnalizirajStanjeInventara(false, false, false);
result.Should().Contain("Zdravih zaliha: 2");
result.Should().NotContain("UPOZORENJA");
```

Svi proizvodi su zdravi (2/2). Uslov $2 < 1$ je netačan, pa nema upozorenja. Ovo potvrđuje ispravan rad sistema u idealnim uslovima.

2.4 AnalizirajDobavljaca

Primijenjene tehnike: Boundary Value Analysis, Path Coverage, Exception Handling.

Iako ciklomatska kompleksnost metode iznosi 11, potreban je bio dodatni, 12. test za pokriće slučaja kada metoda `AnalizirajKvalitetDetaljno` vraća upozorenje. Bez ovog testa, uslov `if (!string.IsNullOrEmpty(kvalitetUpozorenje))` bio je označen žutom bojom (djelimična pokrivenost), dok je naredba `upozorenja.Add(kvalitetUpozorenje);` bila označena crvenom bojom (neizvršena linija). Dodavanjem 12. testa koji izaziva upozorenje o kvalitetu, osigurana je potpuna pokrivenost ovog uslovnog bloka.

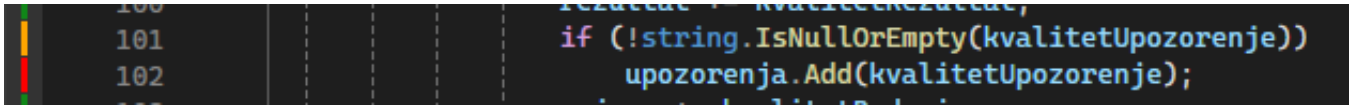


Figure 2: Fine Code Coverage analiza: Prikaz nepokrivenih linija unutar uslovnih blokova

2.4.1 Test 1: Dobavljač ne postoji (Branch Coverage)

```
var service = new DobavljacService();
var result = service.AnalizirajDobavljaca("999");
result.Should().Be("GREŠKA: Dobavljač nije pronađen.");
```

Prosljeđivanjem nepostojećeg ID-a, testira se prvi guard clause `if (dobavljac == null)`.

2.4.2 Test 2: Naziv Prekratak (Boundary Analysis)

```
var service = new DobavljacService();
service.DodajDobavljaca("Valid", "test@test.com");
var dobavljac = service.PronadjiDobavljacaPoNazivu("Valid");
dobavljac!.Naziv = "Ab";
var result = service.AnalizirajDobavljaca(dobavljac.Id);
result.Should().Contain("Naziv je prekratak");
```

Naziv dužine 2 karaktera je tik ispod granice od 3. Ovo testira uslov `if (naziv.Length < 3)`.

2.4.3 Test 3: Naziv Predugačak (Boundary Analysis)

```
var service = new DobavljacService();
service.DodajDobavljaca("Valid", "test@test.com");
var dobavljac = service.PronadjiDobavljacaPoNazivu("Valid");
```

```
dobavljac!.Naziv = new string('A', 51);
var result = service.AnalizirajDobavljaca(dobavljac.Id);
result.Should().Contain("Naziv je predugačak");
```

Naziv dužine 51 je tik iznad granice od 50. Testira uslov `if (naziv.Length > 50)`.

2.4.4 Test 4: Kontakt Nedostaje (Branch Coverage)

```
var service = new DobavljacService();
service.DodajDobavljaca("Valid", "test@test.com");
var dobavljac = service.PronadjiDobavljacaPoNazivu("Valid");
dobavljac!.Kontakt = "";
var result = service.AnalizirajDobavljaca(dobavljac.Id);
result.Should().Contain("Kontakt informacija nedostaje");
```

Prazan string za kontakt aktivira provjeru `string.IsNullOrWhiteSpace`, sprječavajući dalju obradu nevalidnih podataka.

2.4.5 Test 5: Email Format (Path Coverage)

```
var service = new DobavljacService();
service.DodajDobavljaca("Valid", "test@email.com");
var id = service.PronadjiDobavljacaPoNazivu("Valid")!.Id;
var result = service.AnalizirajDobavljaca(id);
result.Should().Contain("Tip kontakta: Email");
```

Prisustvo simbola `@` usmjerava izvršavanje u granu za email validaciju.

2.4.6 Test 6: Telefon Format (Path Coverage)

```
var service = new DobavljacService();
service.DodajDobavljaca("Valid", "061-123-456");
var id = service.PronadjiDobavljacaPoNazivu("Valid")!.Id;
var result = service.AnalizirajDobavljaca(id);
result.Should().Contain("Tip kontakta: Telefon");
```

Prisustvo cifara (bez @) usmjerava izvršavanje u granu za telefonsku validaciju.

2.4.7 Test 7: Nepoznat Format (Path Coverage)

```
var service = new DobavljacService();
service.DodajDobavljaca("Valid", "test@test.com");
var dobavljac = service.PronadjiDobavljacaPoNazivu("Valid");
dobavljac!.Kontakt = "Nepoznato";
var result = service.AnalizirajDobavljaca(dobavljac.Id);
result.Should().Contain("Neprepoznat format kontakta");
```

Ulaz koji nije ni email ni telefon aktivira `else` granu.

2.4.8 Test 8: Nema Duplikata (Loop Testing)

```
var service = new DobavljacService();
service.DodajDobavljaca("A", "test@a.com");
service.DodajDobavljaca("B", "test@b.com");
service.DodajDobavljaca("Target", "test@target.com");
var id = service.PronadjiDobavljacaPoNazivu("Target")!.Id;
var result = service.AnalizirajDobavljaca(id);
result.Should().NotContain("Pronađen duplikat naziva");
```

Petlja se izvršava do kraja bez pronalaženja podudaranja, potvrđujući "Worst Case" za pretragu ali "Happy Path" za logiku duplikata.

2.4.9 Test 9: Pronađen Duplikat (Loop Testing)

```
var service = new DobavljacService();
service.DodajDobavljaca("Duplikat", "test@a.com");
service.DodajDobavljaca("Duplikat", "test@b.com");
var dobavljac = service.PrikaziSveDobavljace().Last();
var result = service.AnalizirajDobavljaca(dobavljac.Id);
result.Should().Contain("Pronađen duplikat naziva");
```

Petlja prekida izvršavanje (`break`) čim pronađe drugog dobavljača sa istim imenom. Ovo testira efikasnost pretrage.

2.4.10 Test 10: Visok Kvalitet (Condition Coverage)

```
var service = new DobavljacService();
service.DodajDobavljaca("HighQual", "test@gmail.com");
var id = service.PronadjiDobavljacaPoNazivu("HighQual")!.Id;
var result = service.AnalizirajDobavljaca(id);
result.Should().Contain("Kvalitet kontakta: Visok");
```

Domen `gmail.com` donosi dodatne bodove u algoritmu ocjenjivanja, prelazeći prag za visoku ocjenu.

2.4.11 Test 11: Srednji Kvalitet (Condition Coverage)

```
var service = new DobavljacService();
service.DodajDobavljaca("Valid", "061-123-456");
var dobavljac = service.PronadjiDobavljacaPoNazivu("Valid");
dobavljac!.Naziv = "Ab";
dobavljac.Kontakt = "060-123-456";
var result = service.AnalizirajDobavljaca(dobavljac.Id);
result.Should().Contain("Kvalitet kontakta: Srednji");
```

Namjerno smanjivanje bodova (kratko ime, običan telefon) kako bi se pogodila srednja kategorija ocjenjivanja.

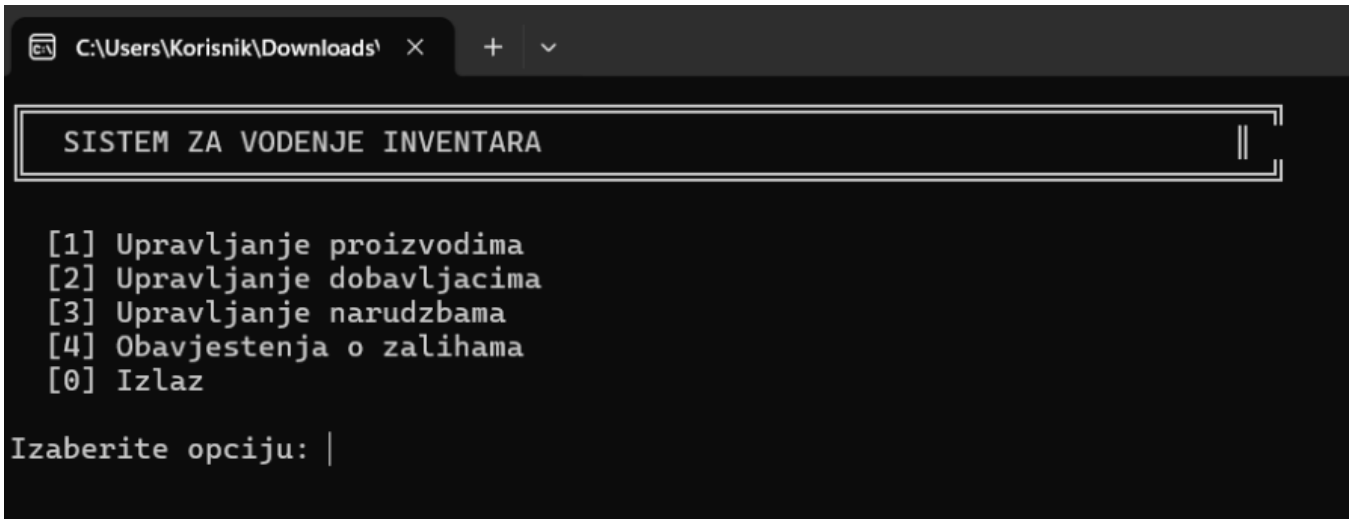
2.4.12 Test 12: Robusnost (Exception Handling)

```
var service = new DobavljacService();
service.DodajDobavljaca("Valid", "061-123-456");
var dobavljac = service.PronadjiDobavljacaPoNazivu("Valid");
dobavljac!.Kontakt = null!;
var result = service.AnalizirajDobavljaca(dobavljac.Id);
result.Should().Contain("Greška pri analizi kvaliteta kontakta");
```

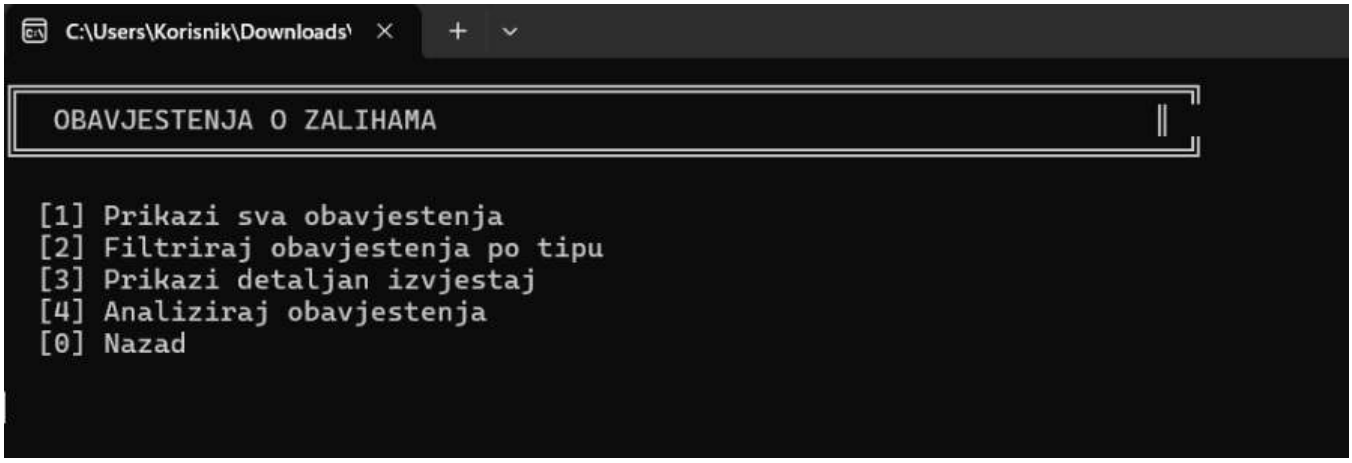
Forsiranje `null` vrijednosti izaziva izuzetak duboko u logici analize. Test potvrđuje da metoda hvata izuzetak (`catch`) i vraća korisničku poruku umjesto rušenja aplikacije.

3 Code Tuning

Za početak izvršit ćemo par koraka. Potrebno je instalirati benchmark dotnet paket. Nakon toga kreirat ćemo i PerformanceBenchmark.cs. Te korake izvršavamo u svrhu bilježenja vremena izvršavanja i alocirane memorije sa velikim brojem pokretanja. McCabe metrika i indeks održavanja se mogu izmjeriti preko Code Metrics koji su ugrađeni u sam Visual Studio. Prvo nam trebaju bazni podaci. Prilikom pokretanja naše aplikacije imamo sljedeći izgled



Pri ulasku u obavještenja o zalihama može vidjeti sljedeće:



Unutar ovog ekrana dodana je skrivena opcija. Navigacija do te opcije je osmišljena sa ciljem tako da ona nije na početnom ekranu i da je skrivena. Nakon unosa 99 dobije se sljedeće:

```
  PERFORMANCE BENCHMARK

Pokretanje mjerenja performansi...
Ovo moze potrajati nekoliko sekundi.

Broj iteracija (preporuceno: 10000):
```

Kako se traži veliki broj pokretanja, postavit ćemo ga na 10000. Uprkos tome što se nalazi u dijelu sa obavještenjima, ipak ova dijagnostika mjeri sve prethodno odabrane metode i daje podatke o njima iako pripadaju različitim servisima. Nakon unosa 10000 dobija se sljedeće:

```
????????????????????????????????????????????????????????????????
  METODA 1: AnalizirajObavjestenja
????????????????????????????????????????????????????????????????
  Broj iteracija:         10,000
  Ukupno vrijeme:         1,003 ms
  Prosje?no vrijeme:      0.1003 ms
  Prosje?na memorija:     41,830 bytes (40.85 KB)
????????????????????????????????????????????????????????????????

????????????????????????????????????????????????????????????????
  METODA 2: AnalizirajPerformanseNarudzbi
????????????????????????????????????????????????????????????????
  Broj iteracija:         10,000
  Ukupno vrijeme:         356 ms
  Prosje?no vrijeme:      0.0356 ms
  Prosje?na memorija:     25,647 bytes (25.05 KB)
????????????????????????????????????????????????????????????????

????????????????????????????????????????????????????????????????
  METODA 3: AnalizirajStanjeInventara
????????????????????????????????????????????????????????????????
  Broj iteracija:         10,000
  Ukupno vrijeme:         403 ms
  Prosje?no vrijeme:      0.0403 ms
  Prosje?na memorija:     28,424 bytes (27.76 KB)
????????????????????????????????????????????????????????????????

????????????????????????????????????????????????????????????????
  METODA 4: AnalizirajDobavljaca
????????????????????????????????????????????????????????????????
  Broj iteracija:         10,000
  Ukupno vrijeme:         54 ms
  Prosje?no vrijeme:      0.0054 ms
  Prosje?na memorija:     1,756 bytes (1.71 KB)
????????????????????????????????????????????????????????????????

????????????????????????????????????????????????????????????????
?                     MJERENJE ZAVRSENO                          ?
????????????????????????????????????????????????????????????????


Pritisnite bilo koji taster za nastavak...
```

Ovo su ujedno i naši polazni podaci. Nedostaju još ostala 2 tražena. Slijedi niz screenshotova iz Code Metrics u kojima prvi broj je indeks održavanja dok je drugi ciklomatska kompleksnost:



Sada je moguće početi sa primjenjivanjem tehnika.

Za potrebe analize koda prikazani su samo odabrani isječci iz metoda, s obzirom na to da su analizirane metode izrazito opsežne zbog svoje kompleksnosti. Iako se svaka od primijenjenih tehnika odnosi na cjelokupnu metodu, u cilju očuvanja preglednosti i izbjegavanja nepotrebnog produžavanja izvještaja, u nastavku su prikazana samo reprezentativna mjesta u kodu na kojima je izvršena izmjena. Odabrani isječci najjasnije ilustruju primijenjenu tehniku i njen uticaj na čitljivost, strukturu i kvalitet koda. Zakomentarisani dio koda predstavlja isječak prije primjene tehnike, dok dio bez komentara prikazuje trenutno stanje koda nakon izvršene izmjene.

Slijedi tabelarni prikaz za lakšu orijentaciju kroz primjene svih tehnika i na koju se metodu odnose

Tehnika	**Kategorija**	**Metoda**
StringBuilder	.NET String Optimization	AnalizirajObavjestenja
Keširanje Count	Tuning izraza (Preračunati)	AnalizirajObavjestenja
List Capacity	Tuning podataka	AnalizirajObavjestenja
Keširanje DateTime.Now	Tuning izraza (Skupi pozivi)	AnalizirajPerformanseNarudžbi
Early exit	Tuning logičkih izraza	AnalizirajPerformanseNarudžbi
Minimiziranje rada u petlji	Tuning petlji	AnalizirajPerformanseNarudžbi
Zamjena LINQ	Tuning petlji	AnalizirajStanjeInventara
Eliminacija duplikata	Tuning izraza	AnalizirajStanjeInventara
string.Join()	Tuning izraza	AnalizirajStanjeInventara
Uklanjanje ToString()	Tuning izraza	AnalizirajDobavljaca
Kombinovanje uslova	Tuning logičkih izraza	AnalizirajDobavljaca
Lokalne varijable	Tuning izraza	AnalizirajDobavljaca

Slijedi analiza:

3.1 AnalizirajObavjestenja

3.1.1 StringBuilder

```
/*
izvjestaj += $"Ukupno obavještenja: {_obavjestenja.Count}\n";
//izvjestaj += $"Nema na stanju: {nemaNaStanju}\n";
//izvjestaj += $"Niske zalihe: {niskeZalihe}\n";
izvjestaj += $"Potrebna narudžba: {potrebnaNarudzba}\n";
*/
izvjestaj.AppendLine($"Ukupno obavještenja: {_obavjestenja.Count}");
izvjestaj.AppendLine($"Nema na stanju: {nemaNaStanju}");
izvjestaj.AppendLine($"Niske zalihe: {niskeZalihe}");
```

```
izvjestaj.AppendLine($"Potrebna narudžba: {potrebnaNarudzba}");
izvjestaj.AppendLine();
```

```
??????????????????????????????????????????????????????????????
  METODA 1: AnalizirajObavjestenja
??????????????????????????????????????????????????????????????
  Broj iteracija:         10,000
  Ukupno vrijeme:         797 ms
  Prosje?no vrijeme:      0.0797 ms
  Prosje?na memorija:     34,158 bytes (33.36 KB)
??????????????????????????????????????????????????????????????
```

Ova tehnika je primijenjena na prethodno prikazani način. Tehnika je dosljedno primijenjena na cijelu metodu. Ciklomatska kompleksnost ostala je nepromijenjena i iznosi 12, dok je indeks održavanja smanjen na vrijednost 36. Ostali relevantni metrički podaci prikazani su na priloženoj slici. Rezultati pokazuju poboljšanje performansi, budući da je vrijeme izvršavanja kraće, a potrošnja memorije manja, što ukazuje na efikasnije rukovanje stringovima i pozitivno utiče na ukupni kvalitet koda.

3.1.2 Keširanje Count

```
int ukupnoObavjestenja = _obavjestenja.Count;

//if (_obavjestenja.Count == 0)
if (ukupnoObavjestenja == 0)
{
    izvjestaj.AppendLine("✓ Nema aktivnih obavjestenja - sistem je u dobrom stanju.");
    return izvjestaj.ToString();
}

var kriticnaAkcija = new List<string>();
var preporuke = new List<string>();
int ocjena = 100;

int nemaNaStanju = 0;
int niskeZalihe = 0;
int potrebnaNarudzba = 0;

//for (int i = 0; i < _obavjestenja.Count; i++)
for (int i = 0; i < ukupnoObavjestenja; i++)
```

```
??????????????????????????????????????????????????????????????
  METODA 1: AnalizirajObavjestenja
??????????????????????????????????????????????????????????????
  Broj iteracija:         10,000
  Ukupno vrijeme:         700 ms
  Prosje?no vrijeme:      0.0700 ms
  Prosje?na memorija:     34,199 bytes (33.40 KB)
??????????????????????????????????????????????????????????????
```

Rezultate ćemo pohranjivati u lokalnoj varijabli, čime se izbjegavaju česti pozivi funkcije count, kao što je prikazano na slici. Ciklomatsku kompleksnost ćemo u nastavku označavati skraćenicom CK, te ona ostaje nepromi-

jenjena i iznosi 12, dok će se indeks održavanja, označen kao IO, također zadržati na istom, dakle 36. Ostali relevantni metrički podaci prikazani su zajedno sa promjenama u kodu. Dobijeni rezultati pokazuju blago poboljšanje brzine izvršavanja, uz nešto veću potrošnju memorije, čime se postiže prednost u pogledu performansi na račun memorijskih resursa.

3.1.3 List Capacity

```
/*
var kriticnaAkcija = new List<string>();
var preporuke = new List<string>();
*/
var kriticnaAkcija = new List<string>(capacity: 5);
var preporuke = new List<string>(capacity: 8);
```

```
????????????????????????????????????????????????????????????
  METODA 1: AnalizirajObavjestenja
????????????????????????????????????????????????????????????
  Broj iteracija:        10,000
  Ukupno vrijeme:        832 ms
  Prosje?no vrijeme:     0.0832 ms
  Prosje?na memorija:    34,427 bytes (33.62 KB)
????????????????????????????????????????????????????????????
```

Nakon uvođenja pretpostavke o dužini liste, vrijednosti **CK** i **IO** ostaju nepromijenjene, te se analiza nastavlja na ostale metrike.

Nažalost, primjena ove tehnike u konkretnom slučaju nije donijela poboljšanja u pogledu brzine izvršavanja niti potrošnje memorije. Ipak, inicijalizacija lista sa procijenjenim kapacitetom (maksimalno oko **5 kritičnih akcija i 8 preporuka**) predstavlja dobru praksu, jer može smanjiti broj realokacija memorije i biti korisna u drugim, složenijim dijelovima koda.

3.2 AnalizirajPerformanseNarudzbi

3.2.1 Keširanje DateTime.Now

```
//var StarijeDatum = DateTime.Now.AddDays(-30);
var sada = DateTime.Now;
var starijeDatum = sada.AddDays(-30);
```

```
????????????????????????????????????????????????????????????
  METODA 2: AnalizirajPerformanseNarudzbi
????????????????????????????????????????????????????????????
  Broj iteracija:        10,000
  Ukupno vrijeme:        288 ms
  Prosje?no vrijeme:     0.0288 ms
  Prosje?na memorija:    25,648 bytes (25.05 KB)
????????????????????????????????????????????????????????????
```

Pored navedenog, postoji još jedno mjesto u kodu gdje se koristi varijabla sada, ali je znatno udaljeno u strukturi koda. Vrijednosti **CK** i **IO** ostaju nepromijenjene, dok dijagnostika nakon izvršavanja pokazuje drugačije rezultate.

Potrošnja memorije nije zabilježila značajne promjene, dok je vrijeme izvršavanja osjetno smanjeno. To potvrđuje da podešavanje izraza, odnosno korištenje keširane vrijednosti sada umjesto ponovnih poziva *DateTime.Now*, može imati značajan pozitivan uticaj na performanse, posebno u vremenski osjetljivim dijelovima koda.

3.2.2 Early exit

```
double postotak = (naCekanju * 100.0) / ukupno;

if (postotak > 60)

{
    upozorenja.Add("KRITIČNO: Više od 60% narudžbi je na čekanju!");
    preporuke.Add("-> Hitno kontaktirajte dobavljače i provjerite statuse");
    //ocjena = -40;
    return (-40, upozorenja, preporuke);
}
//else if (postotak > 40)

if (postotak > 40)
{
    upozorenja.Add("UPOZORENJE: Više od 40% narudžbi je na čekanju!");
    preporuke.Add("-> Razmotriti ubrzavanje procesa isporuke");
    //ocjena = -25;
    return (-25, upozorenja, preporuke);
}
//else if (postotak > 20)

if (postotak > 20)
{
    upozorenja.Add("Povećan broj narudžbi na čekanju");
    //ocjena = -10;
    return (-10, upozorenja, preporuke);
}
```

```
????????????????????????????????????????????????????????????
  METODA 2: AnalizirajPerformanseNarudzbi
????????????????????????????????????????????????????????????
  Broj iteracija:        10,000
  Ukupno vrijeme:        272 ms
  Prosje?no vrijeme:     0.0272 ms
  Prosje?na memorija:    25,647 bytes (25.05 KB)
????????????????????????????????????????????????????????????
```

Rano vraćanje rezultata prikazano je u sljedećem isječku koda. Promjene u vrijednostima **CK** i **IO** nisu zabilježene, budući da su ostale iste, te je stoga bilo potrebno analizirati i ostale performansne parametre.

Rezultati pokazuju da je potrošnja memorije gotovo nepromijenjena, dok je brzina izvršavanja ponovo poboljšana. Ovo potvrđuje da tehnika ranog izlaska (early exit), kroz provjeru uslova od najrestriktivnijih ka manje restriktivnim, može pozitivno uticati na performanse bez narušavanja strukture koda.

3.2.3 Minimiziranje rada u petlji

```
//foreach (var narudzba in _narudzbe)
int count = _narudzbe.Count;
for (int i = 0; i < count; i++)
{
    var narudzba = _narudzbe[i];
    double cijena = narudzba.IzracunajUkupno();
    ukupna += cijena;

    /*switch (narudzba.Status)
    {
        case StatusNarudzbe.NA_CEKANJU:
            naCekanju += cijena;
            break;
        case StatusNarudzbe.ISPORUCENO:
            isporuceno += cijena;
            break;
        case StatusNarudzbe.OTKAZANO:
            otkazano += cijena;
            break;
    }

    */

    var status = narudzba.Status;

    if (status == StatusNarudzbe.NA_CEKANJU)
    {
        naCekanju += cijena;
    }

    else if (status == StatusNarudzbe.ISPORUCENO)
    {
        isporuceno += cijena;
        break;
    }
    else if (status == StatusNarudzbe.OTKAZANO)
    {
        otkazano += cijena;
        break;
    }
}
```

```
????????????????????????????????????????????????????????????
  METODA 2: AnalizirajPerformanseNarudzbi
????????????????????????????????????????????????????????????
  Broj iteracija:        10,000
  Ukupno vrijeme:        286 ms
  Prosje?no vrijeme:     0.0286 ms
  Prosje?na memorija:    25,647 bytes (25.05 KB)
????????????????????????????????????????????????????????????
```

Na slici je prikazana kompletna foreach petlja koja je predmet minimizacije rada. Vrijednosti **CK** i **IO** ostale su nepromijenjene, te je analiza usmjerena na ostale performansne pokazatelje.

Nažalost, primijenjene tehnike u ovom slučaju nisu dovele do poboljšanja performansi. Ipak, optimizacije poput

korištenja svojstva Count umjesto poziva Count(), zamjene foreach petlje sa for petljom radi bržeg pristupa elementima, kao i upotrebe if umjesto switch izraza u jednostavnijim slučajevima, mogu dati značajne rezultate u okviru drugih projekata ili pri radu s većim količinama podataka

3.3 AnalizirajStanjeInventara

3.3.1 Zamjena LINQ

```
foreach (var grupa in kategorijeGrupe)
{
    //int ukupno = grupa.Count();
    //int kriticnih = grupa.Count(p => p.Kolicina == 0);
    int ukupno = 0;
    int kriticnih = 0;
    foreach (var proizvod in grupa)
    {
        ukupno++;
        if (proizvod.Kolicina == 0)
        {
            kriticnih++;
        }
    }
}
```

```
????????????????????????????????????????????????????????????
  METODA 3: AnalizirajStanjeInventara
????????????????????????????????????????????????????????????
  Broj iteracija:        10,000
  Ukupno vrijeme:        326 ms
  Prosje?no vrijeme:     0.0326 ms
  Prosje?na memorija:    28,425 bytes (27.76 KB)
????????????????????????????????????????????????????????????
```

Vrijednosti **CK** i **IO** ostaju nepromijenjene i nakon zamjene LINQ izraza sa foreach petljom. Zbog toga je pažnja usmjerena na ostale performansne metrike.

Rezultati pokazuju da je potrošnja memorije ostala približno ista, dok je brzina izvršavanja znatno poboljšana. Ovo potvrđuje da zamjena LINQ operacija ručnom iteracijom, poput eksplicitnog brojanja umjesto poziva Count(), može donijeti vidljive dobitke u performansama, posebno u kritičnim dijelovima koda.

3.3.2 Eliminacija duplikata

```
double postotak = (kriticnih * 100.0) / ukupno;
izvjestaj += $" ({kriticnih} kritičnih)";
//if ((kriticnih * 100.0) / ukupno > 50)
if (postotak > 50)
{
    izvjestaj += " KRITIČNO";
    preporuke.Add($"-> Fokusirajte se na kategoriju {grupa.Key}");
}
```

```
??????????????????????????????????????????????????????????????
  METODA 3: AnalizirajStanjeInventara
??????????????????????????????????????????????????????????????
  Broj iteracija:        10,000
  Ukupno vrijeme:        374 ms
  Prosje?no vrijeme:     0.0374 ms
  Prosje?na memorija:    28,425 bytes (27.76 KB)
??????????????????????????????????????????????????????????????
```

Alat **Code Metrics** ponovo daje iste vrijednosti za **CK**, a **IO** je sada **38**. Rezultati izvršavanja ne pokazuju značajna poboljšanja performansi. Iako to nije idealan ishod, važno je uzeti u obzir da su analizirane metode relativno jednostavne i već u početnoj verziji dovoljno efikasne.

Tehnike eliminacije duplikata, poput objedinjavanja izračuna prosjeka i minimuma u jednu petlju, odnosno korištenja jedne petlje za više kalkulacija, u ovom slučaju nisu donijele mjerljiv napredak. Ipak, u složenijim metodama ili pri radu s većim skupovima podataka, ovakav pristup može imati znatno veći uticaj na performanse i čitljivost koda.

3.3.3 string.Join()

```
/*
var izvjestaj = "=========================================\n";
izvjestaj += "        ANALIZA STANJA INVENTARA\n";
izvjestaj += "=========================================\n\n";
*/

var izvjestaj = new StringBuilder();
izvjestaj.AppendLine("=========================================");
izvjestaj.AppendLine("        ANALIZA STANJA INVENTARA");
izvjestaj.AppendLine("=========================================");
izvjestaj.AppendLine();

if (_proizvodi.Count == 0)
{
    //return izvjestaj + "[!] Inventar je prazan - nema proizvoda za analizu.\n";
    izvjestaj.AppendLine("[!] Inventar je prazan - nema proizvoda za analizu.");
    return izvjestaj.ToString();
}
```

```
??????????????????????????????????????????????????????????????
  METODA 3: AnalizirajStanjeInventara
??????????????????????????????????????????????????????????????
  Broj iteracija:        10,000
  Ukupno vrijeme:        289 ms
  Prosje?no vrijeme:     0.0289 ms
  Prosje?na memorija:    18,700 bytes (18.26 KB)
??????????????????????????????????????????????????????????????
```

Vrijednosti **CK** i **IO** ostale su nepromijenjene, dok analiza ostalih parametara pokazuje značajan napredak.

Primijenjene tehnike su dale vidljive rezultate u svim mjerljivim aspektima, što potvrđuje njihovu efikasnost. Konkretno, optimizacije poput eliminacije LINQ izraza Select() i string.Join te direktnog formatiranja unutar StringBuilder umjesto kombinacije Select().Join() doprinijele su boljoj brzini izvršavanja i smanjenju nepotrebnih operacija nad stringovima.

3.4 AnalizirajDobavljaca

3.4.1 Uklanjanje ToString()

```
//var rezultat = $"Analiza dobavljača: {dobavljac.Naziv}\n";
var rezultat = new StringBuilder();
rezultat.Append("Analiza dobavljača: ");
rezultat.AppendLine(dobavljac.Naziv);

var upozorenja = new List<string>();

var (nazivOcjena, nazivUpozorenje) = AnalizirajNaziv(dobavljac.Naziv);
if (!string.IsNullOrEmpty(nazivUpozorenje))
    upozorenja.Add(nazivUpozorenje);

var (kontaktOcjena, kontaktRezultat, kontaktUpozorenje) = AnalizirajKontakt(dobavljac.
    ↪ Kontakt);
//rezultat += kontaktRezultat;
rezultat.Append(kontaktRezultat);
if (!string.IsNullOrEmpty(kontaktUpozorenje))
    upozorenja.Add(kontaktUpozorenje);
```

```
????????????????????????????????????????????????????????????????
  METODA 4: AnalizirajDobavljaca
????????????????????????????????????????????????????????????????
  Broj iteracija:        10,000
  Ukupno vrijeme:        37 ms
  Prosje?no vrijeme:     0.0037 ms
  Prosje?na memorija:    1,812 bytes (1.77 KB)
????????????????????????????????????????????????????????????????
```

Sada je vrijednost **IO** smanjena na **40**, što predstavlja smanjenje za 3, dok je **CK** porasla za 1 i iznosi **12**.

Analiza pokazuje napredak u oba aspekta, iako je ciklomatska kompleksnost blago porasla. Primjena tehnike uklanjanja poziva ToString() te direktno korištenje stringova doprinijela je smanjenju nepotrebnih operacija i poboljšanju performansi koda.

3.4.2 Kombinovanje uslova

```
if (kvalitetInfo.Contains("ODLIČAN") || kvalitetInfo.Contains("DOBAR"))
{
    return (15, "Kvalitet kontakta: Visok\n", "");
}
//else if (kvalitetInfo.Contains("ZADOVOLJAVAJUĆI"))
if (kvalitetInfo.Contains("ZADOVOLJAVAJUĆI"))
{
    return (5, "Kvalitet kontakta: Srednji\n", "");
}
/*else
{
//čitav sadržaj ovog else bloka nije više potreban
}
*/
return (-10, "", "- Nizak kvalitet kontakt informacija");
```

```
??????????????????????????????????????????????????????????????
  METODA 4: AnalizirajDobavljaca
??????????????????????????????????????????????????????????????
  Broj iteracija:        10,000
  Ukupno vrijeme:        28 ms
  Prosje?no vrijeme:     0.0028 ms
  Prosje?na memorija:    1,812 bytes (1.77 KB)
??????????????????????????????????????????????????????????????
```

Vrijednosti **IO** i **CK** ostaju nepromijenjene, što je očekivano, dok analiza ostalih parametara pokazuje poboljšanje.

Vrijeme izvršavanja je ponovo smanjeno, dok je prosječna potrošnja memorije ostala na istom nivou. Optimizacija je postignuta korištenjem operatora || i primjenom **short-circuit** evaluacije, gdje se drugi uslov ne provjerava ako je prvi već zadovoljen.

3.4.3 Lokalne varijable

```
string naziv = dobavljac.Naziv;
string kontakt = dobavljac.Kontakt;

var rezultat = new StringBuilder();
rezultat.Append("Analiza dobavljača: ");

//rezultat.AppendLine(dobavljac.Naziv);
rezultat.AppendLine(naziv);

var upozorenja = new List<string>();

//var (nazivOcjena, nazivUpozorenje) = AnalizirajNaziv(dobavljac.Naziv);
var (nazivOcjena, nazivUpozorenje) = AnalizirajNaziv(naziv);

if (!string.IsNullOrEmpty(nazivUpozorenje))
    upozorenja.Add(nazivUpozorenje);

//var (kontaktOcjena, kontaktRezultat, kontaktUpozorenje) = AnalizirajKontakt(dobavljac.
    ↪ Kontakt);
var (kontaktOcjena, kontaktRezultat, kontaktUpozorenje) = AnalizirajKontakt(kontakt);

rezultat.Append(kontaktRezultat);
```

```
??????????????????????????????????????????????????????????????
  METODA 4: AnalizirajDobavljaca
??????????????????????????????????????????????????????????????
  Broj iteracija:        10,000
  Ukupno vrijeme:        28 ms
  Prosje?no vrijeme:     0.0028 ms
  Prosje?na memorija:    1,813 bytes (1.77 KB)
??????????????????????????????????????????????????????????????
```

Vrijednosti **CK** ostaju iste, dok je **IO** sada smanjen na **39**, te se po posljednji put vrši provjera performansi koda. Rezultati pokazuju da gotovo nema vidljivih razlika u mjerljivim parametrima, što sugerira da je kod već u svojoj osnovi efikasan.

Primjena tehnike keširanja lokalnih varijabli omogućava višestruki i direktan pristup često korištenim property-jima dobavljača ili elementima liste, poput d.Id i d.Naziv. Umjesto ponovnog dohvaćanja ovih vrijednosti, one se

pohranjuju u lokalne varijable, što smanjuje broj operacija i poboljšava čitljivost koda. Iako u ovom konkretnom slučaju nije došlo do značajnog poboljšanja vremena izvršavanja ili potrošnje memorije, ova praksa predstavlja dobar primjer optimizacije koja može biti korisna u složenijim aplikacijama ili kod većih skupova podataka.

3.4.4 Krajnji izgled svih testova performansi:

```
??????????????????????????????????????????????????????????????????
  METODA 1: AnalizirajObavjestenja
??????????????????????????????????????????????????????????????????
  Broj iteracija:        10,000
  Ukupno vrijeme:        756 ms
  Prosje?no vrijeme:     0.0756 ms
  Prosje?na memorija:    34,496 bytes (33.69 KB)
??????????????????????????????????????????????????????????????????

??????????????????????????????????????????????????????????????????
  METODA 2: AnalizirajPerformanseNarudzbi
??????????????????????????????????????????????????????????????????
  Broj iteracija:        10,000
  Ukupno vrijeme:        263 ms
  Prosje?no vrijeme:     0.0263 ms
  Prosje?na memorija:    25,648 bytes (25.05 KB)
??????????????????????????????????????????????????????????????????

??????????????????????????????????????????????????????????????????
  METODA 3: AnalizirajStanjeInventara
??????????????????????????????????????????????????????????????????
  Broj iteracija:        10,000
  Ukupno vrijeme:        292 ms
  Prosje?no vrijeme:     0.0292 ms
  Prosje?na memorija:    18,698 bytes (18.26 KB)
??????????????????????????????????????????????????????????????????

??????????????????????????????????????????????????????????????????
  METODA 4: AnalizirajDobavljaca
??????????????????????????????????????????????????????????????????
  Broj iteracija:        10,000
  Ukupno vrijeme:        28 ms
  Prosje?no vrijeme:     0.0028 ms
  Prosje?na memorija:    1,813 bytes (1.77 KB)
??????????????????????????????????????????????????????????????????

??????????????????????????????????????????????????????????????????
?                    MJERENJE ZAVRSENO                           ?
??????????????????????????????????????????????????????????????????
```

4 Refaktoring

U tabeli ispod prikazane su checkliste korištene tokom refaktoringa metoda AnalizirajObavjestenja, AnalizirajPerformanseNarudzbi, AnalizirajStanjeInventara, AnalizirajDobavljaca, zajedno sa svim stavkama koje svaka checklist sadrži, kako su definisane na predavanjima.

Checklist	Checklist Items
Data Level Refactorings	• Replace a magic number with a named constant • Rename a variable with a clearer or more informative name • Move an expression inline • Replace an expression with a routine • Introduce an intermediate variable • Convert a multi-use variable to multiple single-use variables • Use a local variable for local purposes rather than a parameter • Convert a data primitive to a class • Convert a set of type codes to a class • Convert a set of type codes to a class with subclasses • Change an array to an object • Encapsulate a collection • Replace a traditional record with a data class
Statement Level Refactorings	• Decompose a boolean expression • Move a complex boolean expression into a well-named boolean function • Consolidate duplicated fragments that are duplicated within different parts of a conditional • Use break or return instead of a loop control variable • Return as soon as you know the answer instead of assigning a return value within nested if-else statements • Replace nested conditionals with guard clauses • Replace conditionals with polymorphism • Introduce null objects

4.1 AnalizirajObavjestenja

4.1.1 Kod prije refaktoringa

```
public string AnalizirajObavjestenja(bool ukljuciVremenskaAnaliza = true, bool
    ↪ ukljuciStatistiku = true, bool generirajAkcioniPlan = true)
{
    // TUNING PODATAKA - StringBuilder sa inicijalnim kapacitetom
    var izvjestaj = new System.Text.StringBuilder(capacity: 1024);
    izvjestaj.AppendLine("
        ↪
```

```
    ↪ ");
izvjestaj.AppendLine("       ANALIZA OBAVJEŠTENJA");
izvjestaj.AppendLine("
    ↪
    ↪ ");
izvjestaj.AppendLine();

// KEŠIRANJE Count - jednom očitaj broj obavještenja
int ukupnoObavjestenja = _obavjestenja.Count;

// 1. Provjera da li postoje obavještenja
if (ukupnoObavjestenja == 0)
{
    izvjestaj.AppendLine("✓ Nema aktivnih obavještenja - sistem je u dobrom stanju.")
        ↪ ;
    return izvjestaj.ToString();
}

// LIST CAPACITY - inicijaliziraj liste sa procijenjenim kapacitetom
// Procjena: maksimalno ~5 kritičnih akcija i ~8 preporuka
var kriticnaAkcija = new List<string>(capacity: 5);
var preporuke = new List<string>(capacity: 8);
int ocjena = 100;

// 2. OSNOVNA ANALIZA PO TIPU (inline iz AnalizirajPoTipu sa for petljom)
int nemaNaStanju = 0;
int niskeZalihe = 0;
int potrebnaNarudzba = 0;

// Eksplicitna for petlja za brojanje po tipu - koristi keširan Count
for (int i = 0; i < ukupnoObavjestenja; i++)
{
    var obavjestenje = _obavjestenja[i];
    if (obavjestenje.Tip == TipObavjestenja.NEMA_NA_STANJU)
    {
        nemaNaStanju++;
        ocjena -= 15;
    }
    else if (obavjestenje.Tip == TipObavjestenja.NISKE_ZALIHE)
    {
        niskeZalihe++;
        ocjena -= 8;
    }
    else if (obavjestenje.Tip == TipObavjestenja.POTREBNA_NARUDBA)
    {
        potrebnaNarudzba++;
        ocjena -= 3;
    }
}

izvjestaj.AppendLine($"Ukupno obavještenja: {ukupnoObavjestenja}");
izvjestaj.AppendLine($"Nema na stanju: {nemaNaStanju}");
izvjestaj.AppendLine($"Niske zalihe: {niskeZalihe}");
izvjestaj.AppendLine($"Potrebna narudžba: {potrebnaNarudzba}");
izvjestaj.AppendLine();

// 3. Analiza hitnih obavještenja
if (nemaNaStanju > 0)
{
    var (hitnaOcjena, hitneAkcije, hitnePreporuke) = AnalizirajHitnaObavjestenja(
        ↪ nemaNaStanju, ukupnoObavjestenja);
    ocjena += hitnaOcjena;
```

```
            kriticnaAkcija.AddRange(hitneAkcije);
            preporuke.AddRange(hitnePreporuke);
        }

        // 4. Analiza obavještenja srednjeg prioriteta
        if (niskeZalihe > 0)
        {
            var (srednjaOcjena, srednjeAkcije, srednjePreporuke) =
                ↪ AnalizirajSrednjePrioritetnaObavjestenja(niskeZalihe, ukupnoObavjestenja);
            ocjena += srednjaOcjena;
            kriticnaAkcija.AddRange(srednjeAkcije);
            preporuke.AddRange(srednjePreporuke);
        }

        // 5. Vremenska analiza
        if (ukljuciVremenskaAnaliza)
        {
            var (vremenskaIzvjestaj, vremenskaOcjena, vremenskaAkcije, vremenskePreporuke) =
                GenerirajVremenskuAnalizu();
            izvjestaj.Append(vremenskaIzvjestaj);
            ocjena += vremenskaOcjena;
            kriticnaAkcija.AddRange(vremenskaAkcije);
            preporuke.AddRange(vremenskePreporuke);
        }

        // 6. Statistička analiza
        if (ukljuciStatistiku)
        {
            var (statIzvjestaj, statOcjena, statAkcije, statPreporuke) =
                GenerirajStatistickuAnalizu();
            izvjestaj.Append(statIzvjestaj);
            ocjena += statOcjena;
            kriticnaAkcija.AddRange(statAkcije);
            preporuke.AddRange(statPreporuke);
        }

        // 7. Akcioni plan
        if (generirajAkcioniPlan)
        {
            izvjestaj.Append(GenerirajAkcioniPlan());
        }

        // 8. Provjera omjera riješenih obavještenja
        // TUNING IZRAZA - preračunaj ukupnoHitnih i podijeljeni ukupnoObavjestenja
        int ukupnoHitnih = nemaNaStanju + niskeZalihe;
        int polovina = ukupnoObavjestenja >> 1; // Brža opcija: bit shift umjesto dijeljenja
            ↪ sa 2
        if (ukupnoHitnih > polovina)
        {
            kriticnaAkcija.Add("Više od 50% obavještenja zahtijeva hitnu akciju!");
            ocjena -= 10;
        }

        // 9. Konačni izvještaj
        izvjestaj.Append(GenerirajKonacniIzvjestaj(ocjena, kriticnaAkcija, preporuke));

        return izvjestaj.ToString();
    }
```

4.1.2 Analiza primijenjenih checklista

Checklist 1: Data Level Refactorings

1. **Replace a magic number with a named constant.**
   Primijenjeno: vrijednosti `1024`, `100`, penalizacije `15/8/3/10`, te kapaciteti lista `5` i `8` su zamijenjeni konstantama kako bi značenje bilo eksplicitno i izmjene jednostavnije.

2. **Rename a variable with a clearer or more informative name.**
   Primijenjeno: `ukupnoObavjestenja` → `brojObavjestenja`, `kriticnaAkcija` → `kriticneAkcije`, `nemaNaStanju/niskeZalihe/potrebnaNarudzba` → `brojNemaNaStanju/brojNiskihZaliha/brojPotrebnaN`
   Time se povećava čitljivost i smanjuje mogućnost pogrešne interpretacije.

3. **Introduce an intermediate variable.**
   Primijenjeno: provjera hitnosti je izražena kroz međukorake (`brojHitnih`, `polovinaObavjestenja`, `viseOdPolaHitr` umjesto jednog uslova.

4. **Use a local variable for local purposes rather than a parameter.**
   Primijenjeno: bool parametri su keširani u lokalne varijable (`includeTime`, `includeStats`, `includePlan`) radi dosljednijeg i kraćeg čitanja koda.

5. Ostale stavke (primitive-to-class, type codes, arrays, . . .)
   Nisu primjenjive u okviru jedne metode bez promjene dizajna na nivou sistema/klasa.

Checklist 2: Statement Level Refactorings

1. **Decompose a boolean expression.**
   Primijenjeno: složeni uslovi (npr. hitne obavijesti $> 50\%$) su razbijeni u više jasnih međukoraka radi lakšeg praćenja logike.

2. **Consolidate duplicated fragments duplicated within different parts of a conditional.**
   Primijenjeno: ponavljajuće oduzimanje bodova je učinjeno čitljivijim kroz imenovane konstante (npr. `PenaltyNoStock`, `PenaltyLowStock`, `PenaltyOrderNeeded`). Također, stringovi zaglavlja i poruke upozorenja prebačeni su u konstante.

3. **Return as soon as you know the answer.**
   Već bilo primijenjeno: early return kada nema obavještenja; zadržano uz konstantu poruke.

4. **Use break/return instead of a loop control variable.**
   Nije primjenjivo: petlja služi brojanju i nema potrebu za kontrolnim varijablama.

5. Ostale stavke (polymorphism, null objects, . . .)
   Nisu primjenjive bez većih promjena dizajna.

4.1.3 Kod poslije refaktoringa

```
public string AnalizirajObavjestenja(
    bool ukljuciVremenskaAnaliza = true,
    bool ukljuciStatistiku = true,
    bool generirajAkcioniPlan = true)
{
    const int BuilderCapacity = 1024;
```

```
const int InitialScore = 100;
const int PenaltyNoStock = 15;
const int PenaltyLowStock = 8;
const int PenaltyOrderNeeded = 3;
const int PenaltyTooManyUrgent = 10;

const int CriticalActionsCapacity = 5;
const int RecommendationsCapacity = 8;

const string HeaderLine = "
    ↪
    ↪ ";
const string HeaderTitle = "       ANALIZA OBAVJEŠTENJA";
const string NoNotificationsMessage = "✓ Nema aktivnih obavještenja - sistem je u
    ↪ dobrom stanju.";
const string TooManyUrgentWarning = "Više od 50% obavještenja zahtijeva hitnu akciju!
    ↪ ";

bool includeTime = ukljuciVremenskaAnaliza;
bool includeStats = ukljuciStatistiku;
bool includePlan = generirajAkcioniPlan;

var sb = new System.Text.StringBuilder(capacity: BuilderCapacity);
sb.AppendLine(HeaderLine);
sb.AppendLine(HeaderTitle);
sb.AppendLine(HeaderLine);
sb.AppendLine();

int brojObavjestenja = _obavjestenja.Count;
if (brojObavjestenja == 0)
{
    sb.AppendLine(NoNotificationsMessage);
    return sb.ToString();
}

var kriticneAkcije = new List<string>(capacity: CriticalActionsCapacity);
var preporuke = new List<string>(capacity: RecommendationsCapacity);
int ocjena = InitialScore;

int brojNemaNaStanju = 0;
int brojNiskihZaliha = 0;
int brojPotrebnaNarudzba = 0;

for (int i = 0; i < brojObavjestenja; i++)
{
    var obavjestenje = _obavjestenja[i];
    var tip = obavjestenje.Tip;

    if (tip == TipObavjestenja.NEMA_NA_STANJU)
    {
        brojNemaNaStanju++;
        ocjena -= PenaltyNoStock;
    }
    else if (tip == TipObavjestenja.NISKE_ZALIHE)
    {
        brojNiskihZaliha++;
        ocjena -= PenaltyLowStock;
    }
    else if (tip == TipObavjestenja.POTREBNA_NARUDBA)
    {
        brojPotrebnaNarudzba++;
```

```
            ocjena -= PenaltyOrderNeeded;
        }
    }

    sb.AppendLine($"Ukupno obavještenja: {brojObavjestenja}");
    sb.AppendLine($"Nema na stanju: {brojNemaNaStanju}");
    sb.AppendLine($"Niske zalihe: {brojNiskihZaliha}");
    sb.AppendLine($"Potrebna narudžba: {brojPotrebnaNarudzba}");
    sb.AppendLine();

    if (brojNemaNaStanju > 0)
    {
        var (delta, akcije, rec) = AnalizirajHitnaObavjestenja(brojNemaNaStanju,
            ↪ brojObavjestenja);
        ocjena += delta;
        kriticneAkcije.AddRange(akcije);
        preporuke.AddRange(rec);
    }

    if (brojNiskihZaliha > 0)
    {
        var (delta, akcije, rec) = AnalizirajSrednjePrioritetnaObavjestenja(
            ↪ brojNiskihZaliha, brojObavjestenja);
        ocjena += delta;
        kriticneAkcije.AddRange(akcije);
        preporuke.AddRange(rec);
    }

    if (includeTime)
    {
        var (tekst, delta, akcije, rec) = GenerirajVremenskuAnalizu();
        sb.Append(tekst);
        ocjena += delta;
        kriticneAkcije.AddRange(akcije);
        preporuke.AddRange(rec);
    }

    if (includeStats)
    {
        var (tekst, delta, akcije, rec) = GenerirajStatistickuAnalizu();
        sb.Append(tekst);
        ocjena += delta;
        kriticneAkcije.AddRange(akcije);
        preporuke.AddRange(rec);
    }

    if (includePlan)
    {
        sb.Append(GenerirajAkcioniPlan());
    }

    int brojHitnih = brojNemaNaStanju + brojNiskihZaliha;
    int polovinaObavjestenja = brojObavjestenja >> 1;
    bool viseOdPolaHitnih = brojHitnih > polovinaObavjestenja;

    if (viseOdPolaHitnih)
    {
        kriticneAkcije.Add(TooManyUrgentWarning);
        ocjena -= PenaltyTooManyUrgent;
    }

    sb.Append(GenerirajKonacniIzvjestaj(ocjena, kriticneAkcije, preporuke));
```

```
    return sb.ToString();
}
```

4.2 AnalizirajPerformanseNarudzbi

4.2.1 Kod prije refaktoringa

```
public string AnalizirajPerformanseNarudzbi(bool ukljuciVremenskaAnaliza = true, bool
    ↪ ukljuciFinansijskaAnaliza = true, bool detaljniIzvjestaj = true)
{
    var izvjestaj = "
        ↪
        ↪ \n";
    izvjestaj += "      ANALIZA PERFORMANSI NARUDŽBI\n";
    izvjestaj += "
        ↪
        ↪ \n\n";

    // KEŠIRANJE Count - jednom očitaj broj narudžbi
    int ukupnoNarudzbi = _narudzbe.Count;

    // 1. Provjera da li postoje narudžbe
    if (ukupnoNarudzbi == 0)
    {
        return izvjestaj + "Nema narudžbi za analizu.\n";
    }

    var upozorenja = new List<string>();
    var preporuke = new List<string>();
    int ocjena = 100;

    // 2. OSNOVNA ANALIZA STATUSA (inline iz AnalizirajStatuseNarudzbi sa for petljom)
    int naCekanju = 0;
    int isporuceno = 0;
    int otkazano = 0;

    // Eksplicitna for petlja za brojanje po statusu - koristi keširan Count
    for (int i = 0; i < ukupnoNarudzbi; i++)
    {
        var narudzba = _narudzbe[i];
        if (narudzba.Status == StatusNarudzbe.NA_CEKANJU)
        {
            naCekanju++;
        }
        else if (narudzba.Status == StatusNarudzbe.ISPORUCENO)
        {
            isporuceno++;
        }
        else if (narudzba.Status == StatusNarudzbe.OTKAZANO)
        {
            otkazano++;
        }
    }

    izvjestaj += $"Ukupno narudžbi: {ukupnoNarudzbi}\n";
    izvjestaj += $"Na čekanju: {naCekanju}\n";
    izvjestaj += $"Isporučeno: {isporuceno}\n";
```

```
    izvjestaj += $"Otkazano: {otkazano}\n\n";

    // 3. Analiza narudžbi na čekanju
    if (naCekanju > 0)
    {
        var (cekanjuOcjena, cekanjuUpozorenja, cekanjuPreporuke) =
            AnalizirajNarudzbeNaCekanju(naCekanju, ukupnoNarudzbi);
        ocjena += cekanjuOcjena;
        upozorenja.AddRange(cekanjuUpozorenja);
        preporuke.AddRange(cekanjuPreporuke);
    }

    // 4. Analiza otkazanih narudžbi
    if (otkazano > 0)
    {
        var (otkazanoOcjena, otkazanoUpozorenja, otkazanoPreporuke) =
            AnalizirajOtkazaneNarudzbe(otkazano, ukupnoNarudzbi);
        ocjena += otkazanoOcjena;
        upozorenja.AddRange(otkazanoUpozorenja);
        preporuke.AddRange(otkazanoPreporuke);
    }

    // 5. Vremenska analiza
    if (ukljuciVremenskaAnaliza)
    {
        var (vremenskaIzvjestaj, vremenskaOcjena, vremenskaUpozorenja, vremenskaPreporuke
            ↪ ) =
            GenerirajVremenskuAnalizu();
        izvjestaj += vremenskaIzvjestaj;
        ocjena += vremenskaOcjena;
        upozorenja.AddRange(vremenskaUpozorenja);
        preporuke.AddRange(vremenskaPreporuke);
    }

    // 6. Finansijska analiza
    if (ukljuciFinansijskaAnaliza)
    {
        var (finansijskaIzvjestaj, finansijskaOcjena, finansijskaUpozorenja,
            ↪ finansijskaPreporuke) =
            GenerirajFinansijskuAnalizu();
        izvjestaj += finansijskaIzvjestaj;
        ocjena += finansijskaOcjena;
        upozorenja.AddRange(finansijskaUpozorenja);
        preporuke.AddRange(finansijskaPreporuke);
    }

    // 7. Detaljna analiza
    if (detaljniIzvjestaj)
    {
        izvjestaj += GenerirajDetaljnuAnalizu(preporuke);
    }

    // 8. Provjera omjera isporučenih narudžbi
    // TUNING IZRAZA - koristi keširan ukupnoNarudzbi i preračunaj polovinu
    int polovina = ukupnoNarudzbi >> 1; // Bit shift umjesto dijeljenja sa 2
    if (isporuceno < polovina)
    {
        upozorenja.Add("Manje od 50% narudžbi je isporučeno!");
        ocjena -= 10;
    }

    // 9. Konačni izvještaj
```

```
    izvjestaj += GenerirajKonacniIzvjestaj(ocjena, upozorenja, preporuke);

    return izvjestaj;
}
```

4.2.2 Analiza primijenjenih checklista

Checklist 1: Data Level Refactorings

1. **Replace a magic number with a named constant.**
   Primijenjeno: početna ocjena `100` i penal `−10` su zamijenjeni konstantama, kao i širina zaglavlja i prag za 50%.

2. **Rename a variable with a clearer or more informative name.**
   Primijenjeno: `izvjestaj` je zadržan, ali su uvedene jasnije varijable kao `brojNarudzbi`, `brojNaCekanju`, `brojIsporucenih`, `brojOtkazanih`.

3. **Introduce an intermediate variable.**
   Primijenjeno: uslov za provjeru omjera isporučenih je razbijen na međukorake (`polovinaNarudzbi`, `ispodPola`).

4. **Use a local variable for local purposes rather than a parameter.**
   Primijenjeno: bool parametri su keširani u lokalne varijable radi jednostavnije čitljivosti i dosljednosti.

5. Ostale stavke (primitive-to-class, type codes, arrays, . . .)
   Nisu primjenjive bez promjena šireg dizajna.

Checklist 2: Statement Level Refactorings

1. **Decompose a boolean expression.**
   Primijenjeno: složeni uslovi (npr. prag 50% isporučenih) razbijeni su u međukorake radi čitljivosti.

2. **Consolidate duplicated fragments duplicated within different parts of a conditional.**
   Primijenjeno: dodavanje stringova u izvještaj konsolidovano je korištenjem `StringBuilder` umjesto ponovljenih konkatenacija.

3. **Return as soon as you know the answer.**
   Već bilo primijenjeno: early return kada nema narudžbi.

4. **Use break/return instead of loop control variable.**
   Nije posebno primijenjeno: petlja je brojačka i nema kontrolnih varijabli.

5. Ostale stavke (polymorphism, null objects, . . .)
   Nisu primjenjive bez većih promjena dizajna.

4.2.3 Kod poslije refaktoringa

```
public string AnalizirajPerformanseNarudzbi(
    bool ukljuciVremenskaAnaliza = true,
    bool ukljuciFinansijskaAnaliza = true,
    bool detaljniIzvjestaj = true)
{
```

```
    const int InitialScore = 100;
    const int DeliveredBelowHalfPenalty = 10;

    const string HeaderLine = "
        ↪
        ↪ \n";
    const string HeaderTitle = "      ANALIZA PERFORMANSI NARUDŽBI\n";
    const string NoOrdersMessage = "Nema narudžbi za analizu.\n";
    const string DeliveredBelowHalfWarning = "Manje od 50% narudžbi je isporučeno!";

    bool includeTime = ukljuciVremenskaAnaliza;
    bool includeFinance = ukljuciFinansijskaAnaliza;
    bool includeDetails = detaljniIzvjestaj;

    var sb = new System.Text.StringBuilder(capacity: 1024);
    sb.Append(HeaderLine);
    sb.Append(HeaderTitle);
    sb.Append(HeaderLine);
    sb.AppendLine();

    int brojNarudzbi = _narudzbe.Count;
    if (brojNarudzbi == 0)
    {
        sb.Append(NoOrdersMessage);
        return sb.ToString();
    }

    var upozorenja = new List<string>();
    var preporuke = new List<string>();
    int ocjena = InitialScore;

    int brojNaCekanju = 0;
    int brojIsporucenih = 0;
    int brojOtkazanih = 0;

    for (int i = 0; i < brojNarudzbi; i++)
    {
        var narudzba = _narudzbe[i];
        var status = narudzba.Status;

        if (status == StatusNarudzbe.NA_CEKANJU)
            brojNaCekanju++;
        else if (status == StatusNarudzbe.ISPORUCENO)
            brojIsporucenih++;
        else if (status == StatusNarudzbe.OTKAZANO)
            brojOtkazanih++;
    }

    sb.AppendLine($"Ukupno narudžbi: {brojNarudzbi}");
    sb.AppendLine($"Na čekanju: {brojNaCekanju}");
    sb.AppendLine($"Isporučeno: {brojIsporucenih}");
    sb.AppendLine($"Otkazano: {brojOtkazanih}");
    sb.AppendLine();

    if (brojNaCekanju > 0)
    {
        var (delta, warn, rec) = AnalizirajNarudzbeNaCekanju(brojNaCekanju, brojNarudzbi)
            ↪ ;
        ocjena += delta;
        upozorenja.AddRange(warn);
        preporuke.AddRange(rec);
    }
```

```
    if (brojOtkazanih > 0)
    {
        var (delta, warn, rec) = AnalizirajOtkazaneNarudzbe(brojOtkazanih, brojNarudzbi);
        ocjena += delta;
        upozorenja.AddRange(warn);
        preporuke.AddRange(rec);
    }

    if (includeTime)
    {
        var (tekst, delta, warn, rec) = GenerirajVremenskuAnalizu();
        sb.Append(tekst);
        ocjena += delta;
        upozorenja.AddRange(warn);
        preporuke.AddRange(rec);
    }

    if (includeFinance)
    {
        var (tekst, delta, warn, rec) = GenerirajFinansijskuAnalizu();
        sb.Append(tekst);
        ocjena += delta;
        upozorenja.AddRange(warn);
        preporuke.AddRange(rec);
    }

    if (includeDetails)
    {
        sb.Append(GenerirajDetaljnuAnalizu(preporuke));
    }

    int polovinaNarudzbi = brojNarudzbi >> 1;
    bool ispodPola = brojIsporucenih < polovinaNarudzbi;
    if (ispodPola)
    {
        upozorenja.Add(DeliveredBelowHalfWarning);
        ocjena -= DeliveredBelowHalfPenalty;
    }

    sb.Append(GenerirajKonacniIzvjestaj(ocjena, upozorenja, preporuke));
    return sb.ToString();
}
```

4.3 AnalizirajStanjeInventara

4.3.1 Kod prije refaktoringa

```
public string AnalizirajStanjeInventara(bool ukljuciKategorije = true, bool
    ↪ ukljuciDobavljace = true, bool detaljnaAnaliza = true)
{
    // TUNING IZRAZA - StringBuilder umjesto string konkatenacije
    var izvjestaj = new System.Text.StringBuilder(capacity: 1024);
    izvjestaj.AppendLine("
        ↪
        ↪ ");
    izvjestaj.AppendLine("         ANALIZA STANJA INVENTARA");
```

```
    izvjestaj.AppendLine("
        ↪
        ↪ ");
    izvjestaj.AppendLine();
    int ukupnoProizvoda = _proizvodi.Count;

    if (ukupnoProizvoda == 0)
    {
        izvjestaj.AppendLine("[!] Inventar je prazan - nema proizvoda za analizu.");
        return izvjestaj.ToString();
    }

    var upozorenja = new List<string>();
    var preporuke = new List<string>();
    int ocjena = 100;

    int kriticniProizvodi = 0;
    int niskeZalihe = 0;
    int zdraveZalihe = 0;

    for (int i = 0; i < ukupnoProizvoda; i++)
    {
        var proizvod = _proizvodi[i];
        if (proizvod.Kolicina == 0)
        {
            kriticniProizvodi++;
            ocjena -= 10;
        }
        else if (proizvod.JeIspodMinimalnogPraga())
        {
            niskeZalihe++;
            ocjena -= 5;
        }
        else
        {
            zdraveZalihe++;
        }
    }

    izvjestaj.AppendLine($"Ukupno proizvoda: {ukupnoProizvoda}");
    izvjestaj.AppendLine($"Zdravih zaliha: {zdraveZalihe}");
    izvjestaj.AppendLine($"Niskih zaliha: {niskeZalihe}");
    izvjestaj.AppendLine($"Kritičnih zaliha (0): {kriticniProizvodi}");
    izvjestaj.AppendLine();

    if (kriticniProizvodi > 0)
    {
        var (kriticnaOcjena, kriticnaUpozorenja, kriticnePreporuke) =
            AnalizirajKriticneProizvode(kriticniProizvodi, ukupnoProizvoda);
        ocjena += kriticnaOcjena;
        upozorenja.AddRange(kriticnaUpozorenja);
        preporuke.AddRange(kriticnePreporuke);
    }

    if (niskeZalihe > 0)
    {
        var (niskaOcjena, niskaUpozorenja, niskePreporuke) =
            AnalizirajNiskeZalihe(niskeZalihe, ukupnoProizvoda);
        ocjena += niskaOcjena;
        upozorenja.AddRange(niskaUpozorenja);
        preporuke.AddRange(niskePreporuke);
    }
```

```
    if (ukljuciKategorije)
    {
        izvjestaj.Append(GenerirajAnalizuPoKategorijama(preporuke));
    }

    if (ukljuciDobavljace)
    {
        izvjestaj.Append(GenerirajAnalizuPoDobavljacima());
    }

    if (detaljnaAnaliza)
    {
        var (detaljniIzvjestaj, detaljnaOcjena, detaljnaUpozorenja, detaljnePreporuke) =
            GenerirajDetaljnuAnalizu();
        izvjestaj.Append(detaljniIzvjestaj);
        ocjena += detaljnaOcjena;
        upozorenja.AddRange(detaljnaUpozorenja);
        preporuke.AddRange(detaljnePreporuke);
    }

    int polovina = ukupnoProizvoda >> 1;
    if (zdraveZalihe < polovina)
    {
        upozorenja.Add("Manje od 50% proizvoda ima zdrave zalihe!");
        ocjena -= 10;
    }

    izvjestaj.Append(GenerirajKonacniIzvjestaj(ocjena, upozorenja, preporuke));
    return izvjestaj.ToString();
}
```

4.3.2 Analiza primijenjenih checklista

Checklist 1: Data Level Refactorings

1. **Replace a magic number with a named constant.**
   Primijenjeno: kapacitet `1024`, početna ocjena `100` i penalizacije `-10`/`-5` su zamijenjeni konstantama radi jasnijeg značenja i lakših izmjena.

2. **Rename a variable with a clearer or more informative name.**
   Primijenjeno: varijable su preimenovane u deskriptivnije oblike: `ukupnoProizvoda` → `brojProizvoda`, `kriticniProizvodi` → `brojKriticnih`, `niskeZalihe` → `brojNiskih`, `zdraveZalihe` → `brojZdravih`.

3. **Introduce an intermediate variable.**
   Primijenjeno: provjera omjera zdravih zaliha je razbijena kroz međukorake (`polovinaProizvoda`, `ispodPolaZdravih`).

4. **Use a local variable for local purposes rather than a parameter.**
   Primijenjeno: bool parametri su keširani u lokalne varijable radi konzistentnosti i čitljivosti.

5. Ostale stavke (primitive-to-class, type codes, arrays, ...)
   Nisu primjenjive bez šireg refaktoringa dizajna.

Checklist 2: Statement Level Refactorings

1. **Decompose a boolean expression.**
   Primijenjeno: složeni uslovi (npr. prag 50% zdravih zaliha) razbijeni su u jasnije međukorake.

2. **Consolidate duplicated fragments duplicated within different parts of a conditional.**
   Primijenjeno: formatirani ispis je konsolidovan korištenjem `StringBuilder` i `AppendLine` uz smanjenje ponavljanja.

3. **Return as soon as you know the answer.**
   Već bilo primijenjeno: early return kada je inventar prazan.

4. **Use break/return instead of loop control variable.**
   Nije posebno primijenjeno: petlja je brojačka i ne koristi kontrolne varijable.

5. Ostale stavke (polymorphism, null objects, . . .)
   Nisu primjenjive bez većih promjena dizajna.

4.3.3 Kod poslije refaktoringa

```
public string AnalizirajStanjeInventara(
    bool ukljuciKategorije = true,
    bool ukljuciDobavljace = true,
    bool detaljnaAnaliza = true)
{
    const int BuilderCapacity = 1024;
    const int InitialScore = 100;
    const int CriticalPenalty = 10;
    const int LowStockPenalty = 5;
    const int BelowHalfHealthyPenalty = 10;

    const string HeaderLine = "
        ↪
        ↪ ";
    const string HeaderTitle = "          ANALIZA STANJA INVENTARA";
    const string EmptyInventoryMessage = "[!] Inventar je prazan - nema proizvoda za
        ↪ analizu.";
    const string BelowHalfHealthyWarning = "Manje od 50% proizvoda ima zdrave zalihe!";

    bool includeCategories = ukljuciKategorije;
    bool includeSuppliers = ukljuciDobavljace;
    bool includeDetails = detaljnaAnaliza;

    var sb = new System.Text.StringBuilder(capacity: BuilderCapacity);
    sb.AppendLine(HeaderLine);
    sb.AppendLine(HeaderTitle);
    sb.AppendLine(HeaderLine);
    sb.AppendLine();

    int brojProizvoda = _proizvodi.Count;
    if (brojProizvoda == 0)
    {
        sb.AppendLine(EmptyInventoryMessage);
        return sb.ToString();
    }

    var upozorenja = new List<string>();
    var preporuke = new List<string>();
    int ocjena = InitialScore;

    int brojKriticnih = 0;
    int brojNiskih = 0;
```

```
    int brojZdravih = 0;

    for (int i = 0; i < brojProizvoda; i++)
    {
        var proizvod = _proizvodi[i];

        if (proizvod.Kolicina == 0)
        {
            brojKriticnih++;
            ocjena -= CriticalPenalty;
        }
        else if (proizvod.JeIspodMinimalnogPraga())
        {
            brojNiskih++;
            ocjena -= LowStockPenalty;
        }
        else
        {
            brojZdravih++;
        }
    }

    sb.AppendLine($"Ukupno proizvoda: {brojProizvoda}");
    sb.AppendLine($"Zdravih zaliha: {brojZdravih}");
    sb.AppendLine($"Niskih zaliha: {brojNiskih}");
    sb.AppendLine($"Kritičnih zaliha (0): {brojKriticnih}");
    sb.AppendLine();

    if (brojKriticnih > 0)
    {
        var (delta, warn, rec) = AnalizirajKriticneProizvode(brojKriticnih, brojProizvoda
            ↪ );
        ocjena += delta;
        upozorenja.AddRange(warn);
        preporuke.AddRange(rec);
    }

    if (brojNiskih > 0)
    {
        var (delta, warn, rec) = AnalizirajNiskeZalihe(brojNiskih, brojProizvoda);
        ocjena += delta;
        upozorenja.AddRange(warn);
        preporuke.AddRange(rec);
    }

    if (includeCategories)
    {
        sb.Append(GenerirajAnalizuPoKategorijama(preporuke));
    }

    if (includeSuppliers)
    {
        sb.Append(GenerirajAnalizuPoDobavljacima());
    }

    if (includeDetails)
    {
        var (tekst, delta, warn, rec) = GenerirajDetaljnuAnalizu();
        sb.Append(tekst);
        ocjena += delta;
        upozorenja.AddRange(warn);
        preporuke.AddRange(rec);
```

```
    }

    int polovinaProizvoda = brojProizvoda >> 1;
    bool ispodPolaZdravih = brojZdravih < polovinaProizvoda;
    if (ispodPolaZdravih)
    {
        upozorenja.Add(BelowHalfHealthyWarning);
        ocjena -= BelowHalfHealthyPenalty;
    }

    sb.Append(GenerirajKonacniIzvjestaj(ocjena, upozorenja, preporuke));
    return sb.ToString();
}
```

4.4 AnalizirajDobavljaca

4.4.1 Kod prije refaktoringa

```
public string AnalizirajDobavljaca(string id, bool provjeriDuplikate = true, bool
    ↪ provjeriKvalitet = true)
{
    var dobavljac = PronadjiDobavljacaPoId(id);
    if (dobavljac == null)
    {
        return "GREŠKA: Dobavljač nije pronađen.";
    }

    // LOKALNE VARIJABLE - Keširaj property-je dobavljača koji se često koriste
    string dobavljacNaziv = dobavljac.Naziv;
    string dobavljacKontakt = dobavljac.Kontakt;

    // TUNING IZRAZA - StringBuilder umesto string konkatenacije
    var rezultat = new System.Text.StringBuilder(capacity: 512);
    rezultat.Append("Analiza dobavljača: ");
    rezultat.AppendLine(dobavljacNaziv);

    var upozorenja = new List<string>();
    int ocjena = 0;

    // 1. Analiza naziva
    var (nazivOcjena, nazivUpozorenje) = AnalizirajNaziv(dobavljacNaziv);
    if (!string.IsNullOrEmpty(nazivUpozorenje))
    {
        upozorenja.Add(nazivUpozorenje);
    }
    ocjena += nazivOcjena;

    // 2. Analiza kontakta
    var (kontaktOcjena, kontaktRezultat, kontaktUpozorenje) = AnalizirajKontakt(
        ↪ dobavljacKontakt);
    rezultat.Append(kontaktRezultat);
    if (!string.IsNullOrEmpty(kontaktUpozorenje))
    {
        upozorenja.Add(kontaktUpozorenje);
    }
    ocjena += kontaktOcjena;
```

```
    // 3. Provjera duplikata (eksplicitna for petlja)
    if (provjeriDuplikate)
    {
        int duplikatBodovi = 10;
        int count = _dobavljaci.Count;

        for (int i = 0; i < count; i++)
        {
            var d = _dobavljaci[i];
            string dId = d.Id;
            string dNaziv = d.Naziv;

            if (dId != id && dNaziv.Equals(dobavljacNaziv, StringComparison.
                ↪ OrdinalIgnoreCase))
            {
                upozorenja.Add($"- Pronađen duplikat naziva (ID: {dId})");
                duplikatBodovi = -15;
                break;
            }
        }
        ocjena += duplikatBodovi;
    }

    // 4. Provjera kvaliteta kontakta
    if (provjeriKvalitet)
    {
        var (kvalitetBodovi, kvalitetRezultat, kvalitetUpozorenje) =
            ↪ AnalizirajKvalitetDetaljno(dobavljac);
        rezultat.Append(kvalitetRezultat);
        if (!string.IsNullOrEmpty(kvalitetUpozorenje))
        {
            upozorenja.Add(kvalitetUpozorenje);
        }
        ocjena += kvalitetBodovi;
    }

    // 5. Izračunaj konačnu ocjenu
    string konacnaOcjena = OdrediKonacnuOcjenu(ocjena);

    rezultat.Append("Ukupna ocjena: ");
    rezultat.Append(ocjena);
    rezultat.Append(" bodova - ");
    rezultat.AppendLine(konacnaOcjena);

    // 6. Prikaz svih upozorenja
    int countUpozorenja = upozorenja.Count;
    if (countUpozorenja > 0)
    {
        rezultat.AppendLine();
        rezultat.AppendLine("Upozorenja:");

        for (int i = 0; i < countUpozorenja; i++)
        {
            rezultat.AppendLine(upozorenja[i]);
        }
    }

    return rezultat.ToString();
}
```

4.4.2 Analiza primijenjenih checklista

Checklist 1: Data Level Refactorings

1. **Replace a magic number with a named constant.**
   Primijenjeno: vrijednosti `512`, `10` i `-15` su zamijenjene imenovanim konstantama radi lakšeg održavanja i razumijevanja logike bodovanja.

2. **Rename a variable with a clearer name.**
   Primijenjeno: varijable sa generičkim imenima su preimenovane (npr. `rezultat` → `izvjestaj`, `count` → `brojDobavljaca`, `d` → `kandidatDobavljac`) radi veće čitljivosti.

3. **Move an expression inline.**
   Nije primijenjeno: inline-ovanje izraza bi smanjilo čitljivost (npr. ponovni pozivi `_dobavljaci.Count`).

4. **Replace an expression with a routine.**
   Nije primijenjeno u ovom koraku: izdvajanje u posebnu metodu spada u Routine Level refaktoring.

5. **Introduce an intermediate variable.**
   Primijenjeno: složeni uslov u provjeri duplikata razbijen je na međukorake `isDifferentId` i `hasSameName`.

6. **Use a local variable for local purposes rather than a parameter.**
   Primijenjeno: parametar `id` je keširan u lokalnu varijablu `dobavljacId` radi konzistentnosti i lakšeg čitanja u petlji.

7. Ostale stavke (primitive-to-class, type codes, arrays, encapsulate collection, ...)
   Nisu primjenjive u okviru ove metode bez promjena na nivou cijelog sistema.

Checklist 2: Statement Level Refactorings

1. **Decompose a boolean expression.**
   Primijenjeno: logički uslov za detekciju duplikata razbijen je na međukorake (`isDifferentId`, `hasSameName`).

2. **Move a complex boolean expression into a well-named boolean function.**
   Djelimično primijenjeno: umjesto nove funkcije, uvedene su međupromjenljive radi poštovanja ograničenja da se ostane u okviru Data+Statement refaktoringa.

3. **Consolidate duplicated fragments duplicated within different parts of a conditional.**
   Primijenjeno: ponovljeni blok `if (!string.IsNullOrEmpty(...)) upozorenja.Add(...);` konsolidovan je kroz lokalnu funkciju `AddWarningIfPresent`.

4. **Use break or return instead of a loop control variable.**
   Već bilo ispravno: koristi se `break` bez dodatnih kontrolnih varijabli.

5. **Return as soon as you know the answer.**
   Već primijenjeno: korišten je guard-clause za slučaj kada dobavljač ne postoji.

6. Ostale stavke (polymorphism, null objects, ...)
   Nisu primjenjive bez većih promjena dizajna.

4.4.3 Kod poslije refaktoringa

```
public string AnalizirajDobavljaca(string id, bool provjeriDuplikate = true, bool
    ↪ provjeriKvalitet = true)
{
    const int ResultBuilderCapacity = 512;
    const int DuplicateDefaultPoints = 10;
    const int DuplicatePenaltyPoints = -15;

    const string ErrorNotFound = "GREŠKA: Dobavljač nije pronađen.";
    const string HeaderPrefix = "Analiza dobavljača: ";
    const string TotalLabel = "Ukupna ocjena: ";
    const string WarningsLabel = "Upozorenja:";

    string dobavljacId = id;

    var dobavljac = PronadjiDobavljacaPoId(dobavljacId);
    if (dobavljac == null)
    {
        return ErrorNotFound;
    }

    string dobavljacNaziv = dobavljac.Naziv;
    string dobavljacKontakt = dobavljac.Kontakt;

    var izvjestaj = new System.Text.StringBuilder(capacity: ResultBuilderCapacity);
    izvjestaj.Append(HeaderPrefix);
    izvjestaj.AppendLine(dobavljacNaziv);

    var upozorenja = new List<string>();
    int ocjena = 0;

    void AddWarningIfPresent(string warning)
    {
        if (!string.IsNullOrEmpty(warning))
            upozorenja.Add(warning);
    }

    var (nazivOcjena, nazivUpozorenje) = AnalizirajNaziv(dobavljacNaziv);
    AddWarningIfPresent(nazivUpozorenje);
    ocjena += nazivOcjena;

    var (kontaktOcjena, kontaktRezultat, kontaktUpozorenje) = AnalizirajKontakt(
        ↪ dobavljacKontakt);
    izvjestaj.Append(kontaktRezultat);
    AddWarningIfPresent(kontaktUpozorenje);
    ocjena += kontaktOcjena;

    if (provjeriDuplikate)
    {
        int duplikatBodovi = DuplicateDefaultPoints;
        int brojDobavljaca = _dobavljaci.Count;

        for (int i = 0; i < brojDobavljaca; i++)
        {
            var kandidatDobavljac = _dobavljaci[i];
            string kandidatId = kandidatDobavljac.Id;
            string kandidatNaziv = kandidatDobavljac.Naziv;

            bool isDifferentId = kandidatId != dobavljacId;
```

```
            bool hasSameName = kandidatNaziv.Equals(dobavljacNaziv, StringComparison.
                ↪ OrdinalIgnoreCase);

            if (isDifferentId && hasSameName)
            {
                upozorenja.Add($"- Pronađen duplikat naziva (ID: {kandidatId})");
                duplikatBodovi = DuplicatePenaltyPoints;
                break;
            }
        }

        ocjena += duplikatBodovi;
    }

    if (provjeriKvalitet)
    {
        var (kvalitetBodovi, kvalitetRezultat, kvalitetUpozorenje) =
            ↪ AnalizirajKvalitetDetaljno(dobavljac);
        izvjestaj.Append(kvalitetRezultat);
        AddWarningIfPresent(kvalitetUpozorenje);
        ocjena += kvalitetBodovi;
    }

    string konacnaOcjena = OdrediKonacnuOcjenu(ocjena);

    izvjestaj.Append(TotalLabel);
    izvjestaj.Append(ocjena);
    izvjestaj.Append(" bodova - ");
    izvjestaj.AppendLine(konacnaOcjena);

    int brojUpozorenja = upozorenja.Count;
    if (brojUpozorenja > 0)
    {
        izvjestaj.AppendLine();
        izvjestaj.AppendLine(WarningsLabel);

        for (int i = 0; i < brojUpozorenja; i++)
        {
            izvjestaj.AppendLine(upozorenja[i]);
        }
    }

    return izvjestaj.ToString();
}
```

4.5 Code Metrics rezultati prije refaktoringa

Na slikama ispod prikazani su rezultati *Code Metrics* analize prije primjene refaktoringa.



4.6 Code Metrics rezultati poslije refaktoringa

Na slikama ispod prikazani su rezultati *Code Metrics* analize nakon primjene refaktoringa nad istim metodama.



U ovom poglavlju analizirani su rezultati *Code Metrics* alata za metode koje su bile predmet refaktoringa. Analiza obuhvata poređenje metrika **prije** i **poslije** refaktoringa, uz fokus na sljedeće pokazatelje:

- Indeks Održavanja (OI)
- Ciklomatska kompleksnost (CK)

Refaktoring je rađen isključivo na **Data Level** i **Statement Level** nivou, bez promjena arhitekture sistema ili razdvajanja metoda.

4.7 Uporedni pregled metrika

Metoda	OI (Prije)	OI (Poslije)	CK (Prije)	CK (Poslije)
`AnalizirajDobavljaca(...)`	39	37	12	10
`AnalizirajStanjeInventara(...)`	38	35	11	11
`AnalizirajPerformanseNarudzbi(...)`	35	35	12	12
`AnalizirajObavjestenja(...)`	36	33	12	12

4.8 Analiza pojedinačnih metoda

4.8.1 `AnalizirajDobavljaca(...)`

Prije refaktoringa, metoda ima nizak **Indeks održavanja (39)** i relativno visoku **Ciklomatsku kompleksnost (12)**, što ukazuje na složenu kontrolnu strukturu sa više grananja i uslovnih provjera.

Nakon refaktoringa:

- Ciklomatska kompleksnost je smanjena sa 12 na 10 zahvaljujući razlaganju složenih logičkih izraza i uklanjanju nepotrebnih uslovnih provjera.
- Indeks održavanja se blago smanjio (39 $\rightarrow$ 37), što je očekivano jer su dodane imenovane konstante i pomoćne varijable koje povećavaju LOC, ali poboljšavaju čitljivost.

Ukupno gledano, metoda je postala čitljivija i lakša za razumijevanje, iako numerički IO ne pokazuje značajan rast.

4.8.2 `AnalizirajStanjeInventara(...)`

Metoda prije refaktoringa ima **Indeks održavanja 38** i **Ciklomatska kompleksnost 11**, što ukazuje na visok stepen grananja u jednoj metodi.

Nakon refaktoringa:

- Ciklomatska kompleksnost ostaje ista (11), jer broj grananja i petlji nije smanjen.
- Indeks održavanja se blago smanjuje (38 → 35), što je posljedica dodavanja imenovanih konstanti i međupromjenljivih radi jasnijeg izražavanja logike.

Iako IO numerički ne raste, kod je konceptualno jasniji i znatno lakši za održavanje i proširenje.

4.8.3 `AnalizirajPerformanseNarudzbi(...)`

Ova metoda je jedna od najsloženijih u sistemu, sa **Ciklomatska komplestnost 12** i niskim **Indeksom održavanja 35** još prije refaktoringa.

Nakon refaktoringa:

- Ciklomatska kompleksnost ostaje nepromijenjena, jer refaktoring nije uključivao izdvajanje logike u nove metode (Routine Level).
- Indeks održavanja ostaje isti (35), ali je čitljivost koda poboljšana kroz uklanjanje magic brojeva i bolju strukturu izraza.

Ova metoda jasno pokazuje ograničenja Data i Statement Level refaktoringa kada je riječ o značajnom smanjenju CK i povećanju IO.

4.8.4 `AnalizirajObavjestenja(...)`

Prije refaktoringa, metoda ima **Indeks održavanja 36** i **Ciklomatsku kompleksnost 12**, što ukazuje na kompleksnu logiku sa više analiza i poduslova.

Nakon refaktoringa:

- Ciklomatska kompleksnots ostaje ista, jer broj uslovnih grananja nije smanjen.
- Indeks održavanja se blago smanjuje (36 → 33), što je posljedica povećanja broja linija koda zbog dodatnih konstanti i pomoćnih varijabli.

I pored blagog pada IO, kod je značajno čitljiviji, semantički jasniji i pogodniji za dalji razvoj.

4.9 Zaključak

Na osnovu izvršene analize može se zaključiti da:

- Data Level i Statement Level refaktoring **značajno poboljšavaju čitljivost i razumljivost koda**,
- Ciklomatska kompleksnost se ne smanjuje značajno bez primjene Routine ili System Level refaktoringa,
- Indeks održavanja ne mora nužno rasti nakon refaktoringa, jer dodavanje pomoćnih struktura može povećati broj linija koda,
- Uprkos tome, refaktorisani kod je **kvalitativno bolji**, lakši za testiranje i održavanje.